REDRAFT, DON'T JUST DISTILL: REFERENCE-DRIVEN REVISION FOR CONTINUAL MLLM POST-TRAINING

Anonymous authors
Paper under double-blind review

ABSTRACT

Continual post-training of large multimodal models should add new capabilities while preserving those from pre-training, and the two goals pull in opposite directions. SFT gives *explicit target supervision* that learns a task from near-zero accuracy, but its off-policy targets move the model far enough to cause forgetting; on-policy methods such as RLVR and self-distillation preserve *policy proximity* yet supply little signal when the policy cannot yet solve the task. We introduce ReDraft (Reference-Driven Revision and Fine-Tuning), which obtains both from the model's own failures: using an expert response only as a reference, it has the model revise its own incorrect rollout, keeps the revision only if a verifier accepts it, and fine-tunes on what survives. Each retained target is therefore explicit, yet still close to the current policy. Across Counting, Clock Reading, and Jigsaw on Qwen2.5-VL-3B/7B, two of them with near-zero accuracy, ReDraft gains 56.9 points on the target task against SFT's 52.9 while cutting prior-task loss from 16.6 to 1.5 points ($11.3\times$ less forgetting), and improves on OPSD along both axes (19.3 gain, 6.2 loss). Data- and parameter-space analyses match the design: revised targets are more probable under the base model, and the updates they induce stay compact and follow SFT's direction more closely than OPSD's. Together, these results show that revising the model's own rollout rather than directly imitating an expert trajectory can reconcile cold-start acquisition with prior-capability retention.

1 INTRODUCTION

Continual post-training aims to acquire new capabilities while preserving those learned during pre-training (Kirkpatrick et al., 2016; Luo et al., 2023). These goals are often in tension. Effective acquisition, especially for tasks on which the base model has little or no prior competence, requires informative and explicit training targets. In contrast, retaining prior capabilities requires updates that remain close to the model's current behavior. We refer to these two properties as *explicit target supervision* and *policy proximity*, respectively. This raises a central question: *can we provide explicit supervision for cold-start learning while maintaining proximity to the current policy?*

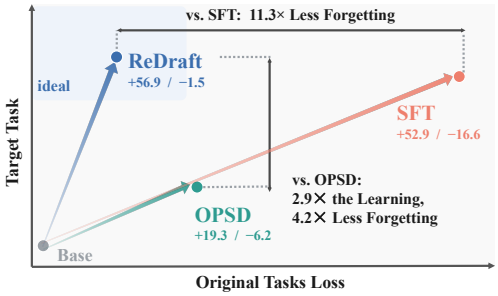


Figure 1: Target-task gain against loss on the original tasks, averaged over all experiments. ReDraft is in the upper-left region.

Existing post-training approaches largely emphasize one of these properties at the expense of the other. Supervised Fine-Tuning (SFT) provides strong *explicit target supervision*: given an expert response $y^\star$, it specifies the desired output at every token, enabling rapid learning even when base-model accuracy is near zero (Wei et al., 2022; Chu et al., 2025). Because expert responses are generated independently of the current policy, however, fitting these off-policy targets may induce large behavioral changes and catastrophic forgetting (Luo et al., 2023; Zhang et al., 2026). Reinforcement Learning with Verifiable Rewards (RLVR) takes the opposite approach, sampling responses from the current policy and scoring them with a verifier; this on-policy training improves *policy proximity* and helps preserve prior capabilities (Lambert et al., 2024; Shenfeld et al., 2025;

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

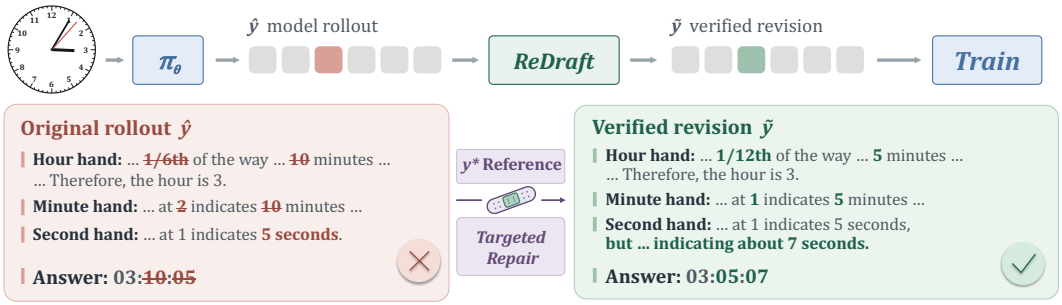


Figure 2: Overview of ReDraft. Each round samples a response, revises incorrect responses using an expert response as a reference, verifies the revisions, and fine-tunes on the resulting targets. By explicitly incorporating the necessary corrections, ReDraft combines *explicit target supervision* with *policy proximity*.

Chen et al., 2025). It therefore refines capabilities that the policy can already partially express (Wu et al., 2026a), but supplies little supervision on a task where almost no sampled response is correct.

Recently, on-policy self-distillation methods such as SDFT (Shenfeld et al., 2026) and OPSD (Zhao et al., 2026) seek to bridge this gap by combining an expert response with trajectories sampled from the current policy (Hübotter et al., 2026). Their correction remains largely implicit, conveyed through a per-token divergence between the reference-conditioned teacher and the student (Lu & Lab, 2025; Hou et al., 2026; Lv et al., 2024): because the two differ only in the reference prefix, the largest divergences often fall on stylistic or pivot tokens rather than on the decisions that settle the task, yielding a dense but potentially misaligned training signal. Per-token clipping can help constrain the resulting policy updates (Zhao et al., 2026; Gu et al., 2024; nrehiew, 2026). Even when a divergence reflects a genuine error, matching the teacher at that position may not suffice to redirect a continuation conditioned on the student's erroneous prefix, so the error may recur later in the response (Jiang & Ferraro, 2026; Fu et al., 2026). This limitation becomes more pronounced at cold start: Self-distillation maintains *policy proximity*, but does not directly provide an explicit corrected trajectory to learn from.

We propose ReDraft (Reference-Driven Revision and Fine-Tuning) to address this limitation by revising the student's rollout into an explicit training target. For each prompt, ReDraft first samples a response from the current policy. Correct responses are kept unchanged, while incorrect responses are revised by the model itself, using an expert response $y^\star$ as a reference. Each revision is then verified, and only successful revisions are used for fine-tuning (Figure 2). Rather than providing token-level corrections along the original, potentially incorrect trajectory, ReDraft externalizes the correction as a complete revised response. Because the revision starts from an on-policy rollout and is generated by the policy itself, the resulting target remains close to the current policy; we therefore call it *self-revised supervision*. This construction combines *explicit target supervision* with *policy proximity*: an incorrect rollout can be converted into a positive training target without requiring the current policy to first discover a correct solution path.

We evaluate ReDraft on Counting (Deitke et al., 2025), Clock Reading (Fu et al., 2025b), and Jigsaw (Wang et al., 2025; Lin et al., 2014) using Qwen2.5-VL-3B and 7B (Bai et al., 2025), where Clock Reading and Jigsaw are genuine cold-start settings with near-zero base-model accuracy. With retention measured over 8 benchmarks of prior capabilities (Section 5), ReDraft exceeds SFT's target-task gain with $11.3\times$ less forgetting, and nearly triples OPSD's gain at under a quarter of its forgetting (Figure 1); the advantage persists under mixed and sequential multi-task training. Further analyses link this result to the two design properties (Section 6): verified revisions have lower perplexity under the base model than SFT targets, consistent with greater *policy proximity*, while in parameter space ReDraft produces compact updates that follow SFT's direction more closely than OPSD's, as *explicit target supervision* predicts. Finally, the ability to self-revise has a scale threshold of its own: when we withhold the target answer from the revision instruction, repairs fall from above 94% at 32B and 72B to under 3% at 3B (Section 7). We therefore propose ReDraft*, where a stronger expert supplies the revision rather than the trajectory, editing what requires correction and leaving the rest of the rollout intact; training on these targets retains most of ReDraft's advantage over SFT.

Our contributions are four-fold:

- We propose ReDraft, a post-training method which converts failed on-policy rollouts into self-revised, verifier-approved targets, combining *explicit target supervision* with *policy proximity*.

- Across three tasks and two model scales, we show that ReDraft matches or exceeds SFT-level acquisition while achieving $11.3\times$ less forgetting, and consistently outperforms OPSD in both target-task acquisition and prior-task retention.
- Data- and parameter-space analyses attribute this advantage to the two design properties: revised targets remain close to the current policy, while the updates they induce follow SFT's direction more closely than OPSD's.
- We show that the model's ability to revise its own rollout scales with model size. By leveraging the revision capability of an external expert model rather than just distilling its trajectory, a 3B model can also acquire a new task while retaining more prior capability than SFT.

2 Related Work

Off-policy and On-policy Post-training. Supervised Fine-tuning (SFT) imitates curated demonstrations token by token (Wei et al., 2022; Yu et al., 2024; Paster et al., 2024); because these trajectories are fixed independently of the learner, it is off-policy and prone to memorization and distribution shift (Ross et al., 2011; Chu et al., 2025). A parallel line of work keeps such targets but clips or reweights per-token updates against a reference policy (Zhu et al., 2025b;a; Liu et al., 2026); ReDraft instead leaves the objective intact and replaces the targets. Reinforcement learning optimizes sampled trajectories from outcome or preference feedback (Schulman et al., 2017; Shao et al., 2024; Guo et al., 2025; Huan et al., 2025) and interferes less with prior capabilities (Shenfeld et al., 2025; Lai et al., 2025; Zhang et al., 2026; Chen et al., 2025), though both benefits presuppose informative rewards. We target the cold-start regime, where SFT acquires the skill but forgets, and RLVR forgets little but cannot bootstrap it.

On-policy Self-distillation. Knowledge distillation transfers a separate teacher's output distribution to a student (Hinton et al., 2015; Kim & Rush, 2016), while on-policy distillation evaluates that guidance along student-sampled trajectories (Agarwal et al., 2024; Xu et al., 2025; Lu & Lab, 2025). Context distillation instead conditions the same model on privileged prompts or demonstrations (Bai et al., 2022; Snell et al., 2022). SDFT distills a demonstration-conditioned teacher on student rollouts, and OPSD conditions the teacher on ground-truth solutions (Shenfeld et al., 2026; Zhao et al., 2026; Yuan et al., 2026), both obtaining dense token-level supervision from on-policy data without a separate teacher. Their correction stays implicit in teacher–student divergence. ReDraft instead writes that correction as a verified response the policy generates itself, keeping the on-policy data benefit without implicit KL credit assignment.

Self-training and Expert-revision. LLM self-training methods generate their own instructions, rationales, or responses and reuse selected outputs for learning (Wang et al., 2023; Chen et al., 2024b; Yuan et al., 2024). STaR (Zelikman et al., 2022) and ReST (Gülçehre et al., 2023) condition rationale generation on answers or hints and fine-tune on filtered trajectories, and self-refinement methods revise responses from model or environment feedback (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024), typically only at inference or by relabelling the learner's states with expert continuations (Lauffer et al., 2025). Three concurrent methods also build targets from the model's own generations, each differing in one respect: ReGFT supplies a partial reference and lets the model complete its own trace, so no failed rollout is edited (Wu et al., 2026b); SPoT has an external oracle do the editing and enforces proximity by surface-form overlap (Lin & Han, 2026); and SD-Zero conditions revision on the binary reward alone, with no reference (He et al., 2026). ReDraft instead edits the policy's own failed response against an expert reference, keeps it only if verified, and regenerates this corpus as training proceeds.

3 Background and Method

3.1 Setup

Problem setting. We post-train a pretrained policy π_0 on a target task $\mathcal{T}$ on which it has little or no initial competence, so the accuracy of its sampled responses is near zero and supervision cannot be obtained by filtering them. We therefore assume two sources of external information: a set of demonstrations $\mathcal{D} = \{(x_i, y_i^\star)\}_{i=1}^N$ that pairs every prompt with an expert reference trajectory $y^\star$,

and a verifier $v(y, y^\star) \in \{0, 1\}$ that judges whether a complete response is correct. Training starts from π_0 and updates the parameters θ; we denote the policy at the current training step by π_θ.

Evaluation axes. Continual post-training is judged on two axes at once: the *gain* on the target task and the *forgetting* on $\mathcal{P}$, a benchmark suite assessing the base model's prior capabilities,

$$G(\theta) = \mathrm{Acc}_{\mathcal{T}}(\pi_\theta) - \mathrm{Acc}_{\mathcal{T}}(\pi_0), \qquad F(\theta) = \mathrm{Acc}_{\mathcal{P}}(\pi_0) - \mathrm{Acc}_{\mathcal{P}}(\pi_\theta), \tag{1}$$

where Acc is accuracy on the corresponding evaluation set.

3.2 Learning from Demonstrations

Supervised fine-tuning (SFT). Supervised fine-tuning imitates the expert trajectory directly:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = \mathbb{E}_{\mathcal{D}}\big[-\log \pi_\theta(y^\star \mid x)\big]. \tag{2}$$

On-policy self-distillation. Rather than fitting $y^\star$ itself, SDFT (Shenfeld et al., 2026) and OPSD (Zhao et al., 2026) place it in the teacher context and distill the reference-conditioned model into the unconditioned one along the student's own rollout. On a rollout $\hat{y}$, let $q_{\theta,n} = \pi_\theta(\cdot \mid x, y^\star, \hat{y}_{<n})$ and $p_{\theta,n} = \pi_\theta(\cdot \mid x, \hat{y}_{<n})$; with forward KL, the objective is

$$\mathcal{L}_{\mathrm{OPD}}(\theta) = \mathbb{E}_{\mathcal{D}}\, \mathbb{E}_{\hat{y}\sim\pi_\theta(\cdot|x)} \frac{1}{|\hat{y}|} \sum_{n=1}^{|\hat{y}|} \mathrm{KL}(q_{\theta,n} \| p_{\theta,n}), \tag{3}$$

with gradients taken through the student branch only.

3.3 Self-Revised Supervision

An ideal training target. Our goal is to improve target-task accuracy while limiting changes to the current policy. For a fixed training pair $(x, y^\star)$, we express this trade-off as maximizing verifier reward with a KL penalty (Schulman et al., 2015):

$$\pi^\star_\beta = \arg\max_{\pi}\ \mathbb{E}_{y\sim\pi(\cdot|x)}\big[v(y, y^\star)\big] - \beta\, D_{\mathrm{KL}}\big(\pi(\cdot \mid x) \,\big\|\, \pi_\theta(\cdot \mid x)\big). \tag{4}$$

The solution for $\beta > 0$ is the tilted distribution $\pi^\star_\beta(y \mid x) \propto \pi_\theta(y \mid x) \exp\big(v(y, y^\star)/\beta\big)$ (Korbak et al., 2022; Rafailov et al., 2023). For a binary verifier, taking $\beta \to 0^+$ gives the ideal target (Appendix A.3):

$$\pi^\star(y \mid x) = \frac{\pi_\theta(y \mid x)\, \mathbf{1}\{y \in \mathcal{C}(x)\}}{Z_\theta(x)}, \qquad Z_\theta(x) = \Pr_{y\sim\pi_\theta(\cdot|x)}\big[y \in \mathcal{C}(x)\big] > 0, \tag{5}$$

where $\mathcal{C}(x) = \{y : v(y, y^\star) = 1\}$ is the set of responses accepted by the verifier. The indicator excludes incorrect responses, so a sample from this target is a complete, verified response for *explicit target supervision*. Among correct responses, the π_θ factor preserves the model's relative preferences rather than concentrating all mass on an external reference. The resulting target is the closest distribution to π_θ, providing a precise notion of *policy proximity*.

Limitations of existing objectives. RLVR and rejection sampling draw from this target but require $1/Z_\theta(x)$ samples per accepted response on average, which becomes costly when $Z_\theta(x) \approx 0$. SFT instead fits $\delta_{y^\star}$, the distribution assigning all probability to the expert response. This provides a correct target but discards the effects of π_θ weighting in Equation 5.

Self-distillation uses the reference-conditioned teacher as a proxy for this ideal target (Shenfeld et al., 2026). To compare their distributions, we write the teacher as a reweighted current policy:

$$\pi_\theta(y \mid x, y^\star) = \pi_\theta(y \mid x)\, w_\theta(y), \qquad w_\theta(y) = \frac{\pi_\theta(y \mid x, y^\star)}{\pi_\theta(y \mid x)}. \tag{6}$$

This identity assumes shared support and requires no further normalization (Appendix A.4). Unlike the correctness indicator in Equation 5, the unverified weight $w_\theta(y)$ need not vanish on incorrect responses. Thus Equation 3 matches teacher probabilities along the student's rollout without constructing an explicit, verified correction.

Consequently, token-level KL may emphasize stylistic rather than task-critical differences: Zhao et al. (2026) reports 6–15$\times$ larger contributions from stylistic than mathematical tokens and uses pointwise clipping to limit them, but clipping neither identifies errors nor provides corrected continuations.

Revision proposal. ReDraft replaces implicit token-level correction with explicit revision and verification of the model's own rollout. It first samples $\hat{y} \sim \pi_\theta(\cdot \mid x)$ and sets $\tilde{y} = \hat{y}$ if the verifier accepts it. Otherwise, the same model generates $\tilde{y}$ from $(x, y^\star, \hat{y})$, with instructions to preserve valid content and correct errors. We denote this keep-or-revise distribution by $R_\theta(\cdot \mid x, y^\star, \hat{y})$.

Assumption 1 (Correction competence)**.** *For a prompt with $Z_\theta(x) < 1$, let $\tilde{y} \sim R_\theta(\cdot \mid x, y^\star, \hat{y})$. The probability of correcting a failed rollout is positive:*

$$r_\theta(x) = \Pr[\tilde{y} \in \mathcal{C}(x) \,|\, \hat{y} \notin \mathcal{C}(x)] > 0, \; \hat{y} \sim \pi_\theta(\cdot \mid x).$$

For each fixed $(x, y^\star)$, averaging over initial rollouts gives the candidate distribution below, with the reference dependence suppressed:

$$\rho_\theta(y \mid x) = \sum_{\hat{y}} \pi_\theta(\hat{y} \mid x)\, R_\theta(y \mid x, y^\star, \hat{y}). \tag{7}$$

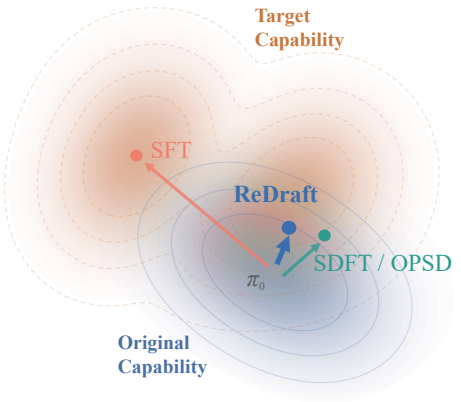


Figure 3: Schematic of where each objective moves the policy. From π_0, SFT reaches the target capability but leaves the original one; SDFT/OPSD keeps the original but barely approaches the target; ReDraft reaches the target without leaving the original.

Among candidates that pass verification, let $\pi_{\text{ReDraft}}(y \mid x)$ denote the probability that the retained response equals y. Conditional probability gives

$$\pi_{\text{ReDraft}}(y \mid x) = \frac{\rho_\theta(y \mid x)\, \mathbf{1}\{y \in \mathcal{C}(x)\}}{S_\theta(x)}, \qquad S_\theta(x) = Z_\theta(x) + \big(1 - Z_\theta(x)\big) r_\theta(x) \;\geq\; Z_\theta(x). \tag{8}$$

Here $S_\theta(x) = \Pr_{\tilde{y} \sim \rho_\theta}[\tilde{y} \in \mathcal{C}(x)] > 0$ is the candidate acceptance rate, and it describes the training targets produced by our procedure. At a fixed number of initial rollouts, revision yields at least as many accepted targets as rejection sampling in expectation: at cold start $Z_\theta(x) \approx 0$, so $S_\theta(x) \approx r_\theta(x)$. Appendix C.1 reports both rates for every task and scale: on the cold-start tasks, initial acceptance is low while repair rates exceed 56%. Examples of revised responses appear in Appendix G.

ReDraft Supervised Fine-tuning. Each retained response becomes a training pair $(x, \tilde{y})$, with $\tilde{y}$ drawn from $\pi_{\text{ReDraft}}(\cdot \mid x)$. The learner predicts this target from x alone, without the reference or initial rollout. Holding the collection process fixed, the expected loss is

$$\mathcal{L}_{\text{ReDraft}}(\theta) = \mathbb{E}_{\mathcal{D}}\Big[S_\theta(x)\, \mathbb{E}_{\tilde{y} \sim \pi_{\text{ReDraft}}(\cdot \mid x)}\big[-\log \pi_\theta(\tilde{y} \mid x) \big] \Big]. \tag{9}$$

The factor $S_\theta(x)$ reflects how often a prompt supplies a retained target, not an additional weight applied to accepted examples.

Every retained target in Equation 9 is verifier-approved. Edited spans provide explicit supervision, while unchanged spans preserve the model's own trajectory and encourage policy proximity.

Summary. Table 1 compares the objectives on the four properties: ReDraft combines SFT's explicit targets supervision with RLVR and OPSD's policy proximity. Figure 3 gives the same picture geometrically: the model moves toward the target capability without imitating SFT's distant expert trajectory or relying on OPSD's implicit KL allocation. This predicts SFT-like acquisition with the retention benefit of on-policy data.

	RLVR	SFT	OPSD	ReDraft
Policy proximity	✓	—	✓	✓
Dense signal	—	✓	✓	✓
Explicit target	✓	✓	—	✓
Usable at zero start	—	✓	—	✓

Table 1: What each supervision method provides. ReDraft pairs policy proximity with explicitly target.

4 Experimental Setup

Task Acquisition. We validate our hypothesis on three canonical vision-language tasks. The base model is already partly competent on *Counting*, whereas its near-zero accuracy on *Jigsaw* and *Clock*

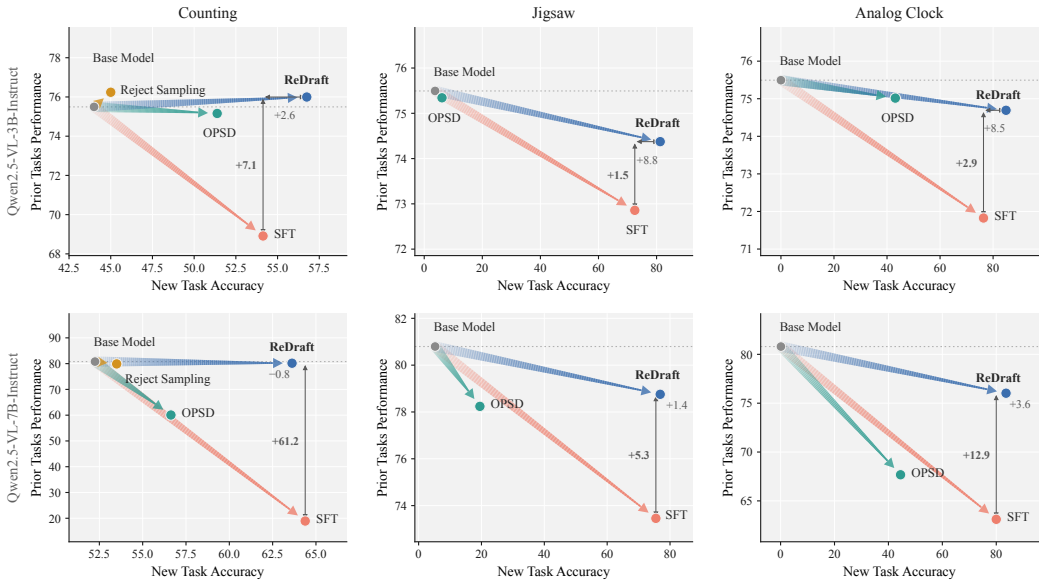


Figure 4: Target-task acquisition vs. retention of prior capabilities. Rows show Qwen2.5-VL-3B/7B; arrows originate at the base model. Reject Sampling is evaluated only on Counting, where the base policy already has substantial competence. Brackets show the ReDraft–SFT difference; higher is better on both axes.

Reading makes these two our cold-start settings. Each task is built from a public image corpus paired with GPT-5.5 expert responses $y^\star$; construction details and hyper-parameters are in Appendix B.

MLLMs. We employ Qwen2.5-VL-3B (Bai et al., 2025) and 7B as our MLLMs due to their strong performance on vision-language understanding and support of native-resolution input.

Evaluation. Beyond the novel tasks, we evaluate the post-trained model on 4 representative capability axes of prior knowledge: (i) OCR, chart & document understanding, covered by AI2D (Kembhavi et al., 2016), DocVQA (Mathew et al., 2021), InfoVQA (Mathew et al., 2022), and ChartQA (Masry et al., 2022); (ii) general VQA, covered by MME (Fu et al., 2025a) and MMStar (Chen et al., 2024a); (iii) real-world spatial understanding, measured by RealWorldQA (xAI, 2024); and (iv) science, measured by ScienceQA (Lu et al., 2022).

5 Results and Analysis

5.1 Main Results

ReDraft preserves SFT-level acquisition while sharply reducing forgetting. ReDraft attains a higher mean target-task gain than SFT (56.9 versus 52.9 points) while reducing prior-task loss from 16.6 to 1.5 points, or 11.3× less forgetting. It leads SFT on target gain in most settings and forgets less in every one (Figure 4). Table 4 in Appendix C.3 reports the detailed results of every experiment. ReDraft therefore matches or slightly improves target-task learning while retaining substantially more original capability.

The advantage is largest when genuine cold start is required. Across Jigsaw and Analog Clock Reading, ReDraft gains 79.4 target-accuracy points on average, vs. 73.8 for SFT and 26.0 for OPSD, while losing only 2.2 prior-task points, versus 7.8 and 4.1. The contrast with OPSD is sharpest at 3B: on Jigsaw it barely leaves its base accuracy, ending at 6.1% target accuracy for a gain of 2.4 points against ReDraft's 77.5, and on Clock Reading it recovers about half of ReDraft's gain (43.1 versus 84.9). Across all settings, ReDraft dominates OPSD and forgets less than SFT in every pair.

RLVR and rejection sampling depend on successful rollouts. We further evaluate the two verifier-based alternatives to expert supervision, GRPO and Reject Sampling (RS). GRPO improves accuracy only on 7B Jigsaw, where correct orderings are occasionally sampled. In the other cold-start settings, correct responses remain scarce and reward increases mainly reflect the format term rather

than task accuracy (Appendix C.2). RS also relies on already-correct samples. We report RS on Counting, where base accuracy is $44.0\%/52.3\%$ for 3B/7B and positive rollouts are readily available. Even there, RS gains only $1.0/1.3$ points, compared with ReDraft's $12.8/11.4$. RS learns only from already-correct rollouts, whereas revision also converts failed attempts into positive training targets.

5.2 Is OPSD simply under-trained?

Because OPSD follows a trade-off direction similar to ReDraft in some settings (Figure 4), we test whether its weaker acquisition results from insufficient training by extending its budget and selecting the best target-task checkpoint. Despite 70% more steps, its mean gain increases by only 4.1 points ($19.3 \rightarrow 23.4$; Table 2). Clock Reading improves the most ($+8.7$), but remains 31.7 points behind ReDraft (52.5 vs. 84.2). This plateau suggests an objective rather than budget limitation: on a new task, without an explicit corrective target, additional steps provide little useful signal. Per-task and per-model results are reported in Appendix C.4.

Table 2: Extending OPSD does not close the gap. Target-task gain in percentage points, averaged over 3B and 7B.

Task	ReDraft	OPSD	
		standard	extended
Counting	$+\mathbf{12.1}$	$+5.9$	$+8.1$
Jigsaw	$+\mathbf{74.6}$	$+8.3$	$+9.6$
Clock Reading	$+\mathbf{84.2}$	$+43.8$	$+52.5$
Mean gain	$+\mathbf{56.9}$	$+19.3$	$+23.4$
Mean steps	367	449	762

5.3 Multi-Task Training

Mixed training. We interleave all three target corpora to test ReDraft without explicit task boundaries (Figure 5). On 3B, ReDraft exceeds SFT in mean target accuracy (62.9 vs. 59.7), with only 0.2 versus 12.3 points of forgetting. On 7B, it nearly matches SFT in target accuracy (68.6 vs. 69.5) while retaining substantially more prior capability (78.4 vs. 31.6).

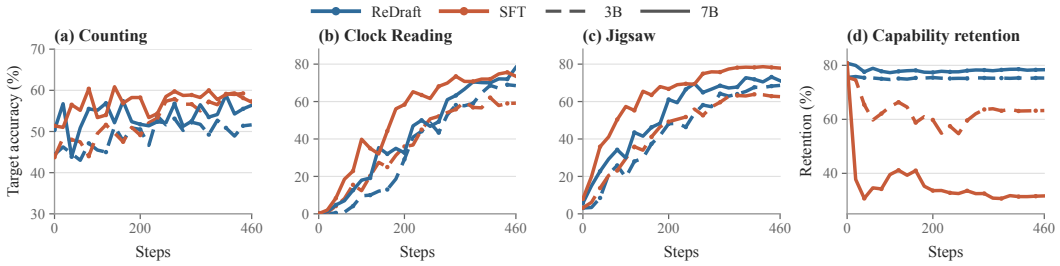


Figure 5: Mixed training at both model scales. All three target corpora are interleaved in a single run. Color identifies the objective and line style the model scale. Panels (a–c) track the three target tasks separately; (d) shows original-task retention over 8 benchmarks.

Sequential training. We further evaluate a Clock→Counting→Jigsaw curriculum (Figure 6). On 3B, ReDraft achieves a higher three-task mean (53.4 vs. 51.0) and retention (74.5 vs. 63.1), although SFT performs better on final-stage Jigsaw (58.1 vs. 50.9). On 7B, ReDraft likewise improves both the three-task mean (66.3 vs. 60.9) and retention (75.9 vs. 33.9).

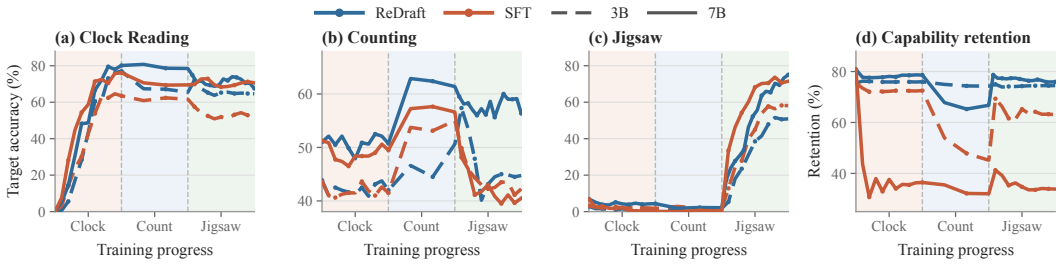


Figure 6: Sequential performance across all three target tasks. Panels (a–c) track Clock Reading, Counting, and Jigsaw throughout the complete curriculum; (d) shows original-task retention. Color identifies the objective and line style the model scale. Backgrounds mark the active training stage.

6 Data- and Parameter-Space Analysis

Since ReDraft matches SFT on the target task while forgetting far less, and dominates OPSD on both axes, we ask where that behavior comes from. This section examines the targets ReDraft trains on and the parameter update they induce. All three objectives start from the same pre-trained weights, so their updates can be compared directly.

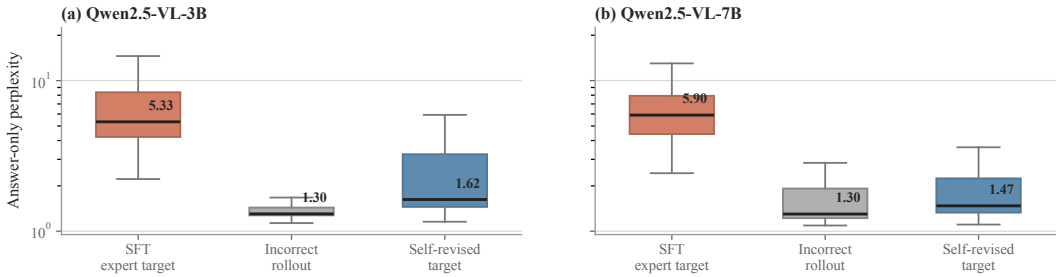


Figure 7: Self-revised targets are more policy-proximal than external SFT targets. Answer-only PPL is computed under the corresponding frozen base model and shown on a logarithmic scale; each box pools 400 paired examples from each of three tasks.

Revised targets retain *policy proximity* in the data. We first examine how likely the training targets are under the generating model, using answer-only perplexity (PPL) under the corresponding frozen base model. For each model and task we score three responses for the same prompts: the external expert trajectory, the model's own incorrect rollout, and its verified self-revision, pooling the three tasks in the aggregate result. Figure 7 shows much lower median PPL for self-revisions than for expert trajectories at both scales (1.62 versus 5.33 on 3B, 70% lower; 1.47 versus 5.90 on 7B, 75% lower). Own rollouts are lowest (1.30 at both scales), providing a reference for the model's original behavior. Low PPL indicates higher likelihood under that model, not task correctness. Verified self-revisions have PPL close to this reference, supporting *policy proximity*, while verification separately establishes correctness.

***Explicit target supervision* steers the update in SFT's direction.** We next examine whether ReDraft's explicit targets induce updates aligned with SFT's. Training each objective from a shared initialization, we take the cosine $\cos(\Delta_A, \Delta_B)$ between the updates of two objectives, per module group. Figure 8 localizes the answer: vision blocks are indistinguishable (0.05–0.06), language-model linear layers favor ReDraft (+0.24 vs. +0.13), and the vocabulary tensors show the largest gap (+0.76 vs. +0.39). Over all parameters $\cos(\Delta_{\text{SFT}}, \Delta_{\text{ReDraft}}) = +0.272$ against +0.135 for SFT–OPSD. This stronger alignment, especially in the vocabulary tensors, is consistent with the explicit token targets shared by ReDraft and SFT. Because the absolute cosines remain modest, this result supports stronger relative alignment with SFT rather than equivalence between the updates; paired results are reported in Appendix E.2.

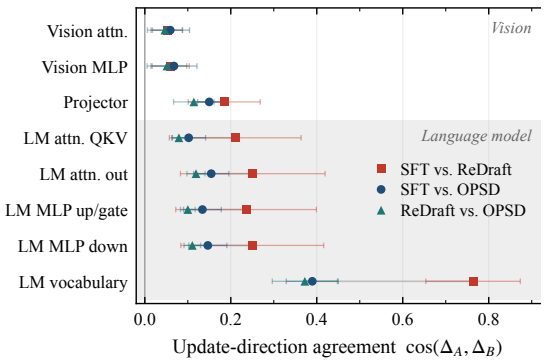


Figure 8: Agreement between the update directions of different methods, per module group. Markers are means over all pairs.

The same intermediate position holds in magnitude and spread: ReDraft moves the weights less than SFT but more than OPSD, and packs that larger update into fewer directions than OPSD rather than spreading it more widely (Appendix E). Token-level inspection further shows that OPSD assigns most credit to incidental reasoning and formatting rather than decisive answer tokens, sometimes pushing the latter in the wrong direction (Appendix F).

7 Revision Capability

Scaling of Self-revision. ReDraft requires the model to repair its own rollout from a reference (Assumption 1). Because the instruction used above reveals the target answer, a shortcut that STaR (Zelikman et al., 2022) also warns for reasoning unfaithful, we test a harder prompt that supplies only the expert trajectory and asks the model to produce a correct revision (Appendix I). Under the simplified prompt the scales separate sharply (Figure 9): the verifier accepts 0.6% and 2.5% of 3B revisions on Clock Reading and Jigsaw, against 17.2% and 18.8% at 7B and 99.5% and 96.8% at 32B, after which the rate saturates. At 3B the revision usually returns the original reading unchanged. First-attempt accuracy rises only from 0.0% at 3B to 0.9% at 72B on Clock Reading, and from 3.3% to 13.5% on Jigsaw, so what scales here is the ability to revise against a reference, not the ability to solve the task.

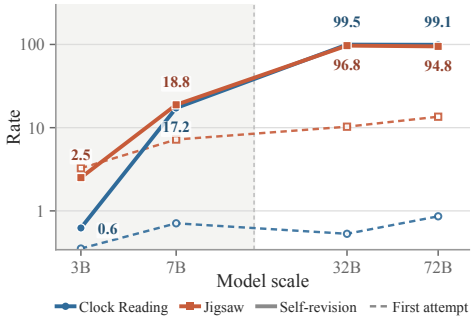


Figure 9: Revision success across model scales under the simplified instruction. Solid lines show successful revisions of failed rollouts; dashed lines show first-attempt accuracy. Shading marks scales where revision mostly fails.

Leveraging the revision capability of an external expert. How should a model that cannot yet revise itself be cold-started? Here we keep the revision step and move it outside the model: the same expert that supplies $y^\star$ is asked to edit the 3B rollout, changing only what requires correction and leaving the rest of the trajectory untouched. The expert is used for its ability to revise rather than as a trajectory to imitate, so as much of the model's own response survives, and we call this variant ReDraft*. Figure 10a shows that pooled over the two tasks, their median answer-only PPL is 3.29, against 8.16 for the expert's own trajectory and 1.75 for the model's own rollout. An externally revised target is therefore far closer to the model's distribution than the expert trace it would otherwise imitate, though not as close as a revision the model writes itself. Training on these targets recovers most of ReDraft's behavior (Figure 10b,c): on Clock Reading ReDraft* reaches 85.8 target accuracy against SFT's 76.4 while retaining 72.5 against 71.8, and on Jigsaw it reaches 79.3 against 72.5 while retaining 75.0 against 72.9, the least forgetting of the three objectives. The 3B model therefore acquires the new task while retaining more of its prior capability than SFT allows.

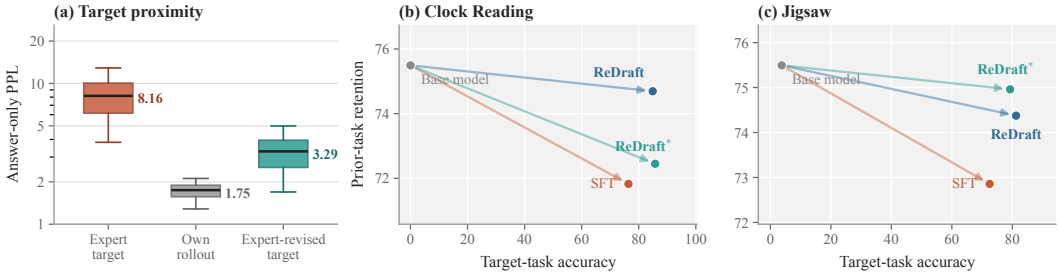


Figure 10: Expert revision of a 3B rollout. (a) Answer-only PPL under Qwen2.5-VL-3B, pooled over Clock Reading and Jigsaw. (b–c) Target-task accuracy against prior-task retention, with arrows originating at the base model; ReDraft* denotes the variant whose revisions are produced by the external expert rather than by itself.

8 Conclusion

We study continual post-training in cold-start settings, where off-policy demonstrations provide explicit targets but induce forgetting, while on-policy self-distillation preserves policy proximity but offers only implicit token-level corrections. ReDraft bridges this gap by converting failed rollouts into verified revised targets that remain close to the current policy. Across three tasks, two model scales, and mixed and sequential training, it matches SFT-level acquisition with much less forgetting and outperforms OPSD in both acquisition and retention. When self-revision fails at smaller scales, we propose ReDraft*: a stronger expert revises the model's rollout while preserving it where possible, enabling cold-start acquisition with better retention than SFT.

AI USAGE STATEMENT

This article employs large language models solely for polishing the sentence structures to better align with standard English writing conventions, with no influence over the research design, experimental implementation or analysis. We review all AI-assisted outputs and take responsibility for the final content of this work.

ETHICS STATEMENT

This work studies continual post-training of multimodal language models: the model repairs its own failed rollouts against an expert reference, and only revisions accepted by an automatic verifier are used for training. All experiments rely on publicly available image corpora and benchmarks and involve no human subjects.

REPRODUCIBILITY STATEMENT

We have described our theory analysis and the experiment setup in Sec. 3 and Sec. 4. To support reproducibility, we will open-source our code.

LIMITATIONS

ReDraft rests on the learner's correction competence (Assumption 1), which holds comfortably here (Appendix C.1) but remains empirical. If the learner cannot revise reliably, no self-revised targets can be produced. Nevertheless, we propose ReDraft* as a more effective cold-start alternative: a stronger expert revises the learner's rollout rather than providing a trajectory for direct distillation. This mitigates but does not eliminate the assumption, since a task that defeats even expert-assisted repair would still leave no usable targets. ReDraft also requires an automatic verifier, so open-ended generation would need a learned or human proxy whose errors enter the targets directly. Each failed rollout costs an extra generation pass. We construct the revised set once from the policy that begins training, so the targets become staler as the parameters move; online revision should restore proximity but remains untested. Finally, we represent the self-distillation family by OPSD as formulated in Equation 3, without its pointwise clipping, and do not evaluate SDFT separately.

REFERENCES

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=3zKtaqxLhW.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. *CoRR*, abs/2502.13923, 2025. doi: 10.48550/ARXIV.2502.13923. URL https://doi.org/10.48550/arXiv.2502.13923.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosiute, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemí Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: harmlessness

from AI feedback. *CoRR*, abs/2212.08073, 2022. doi: 10.48550/ARXIV.2212.08073. URL https://doi.org/10.48550/arXiv.2212.08073.

Yuchen Cai, Ding Cao, Liang Lin, Chunxi Luo, Xin Xu, Kai Yang, Weijie Liu, Saiyong Yang, Tianxiang Zhao, Guangzhong Sun, Guiquan Liu, and Junfeng Fang. Learning to foresee: Unveiling the unlocking efficiency of on-policy distillation. *CoRR*, abs/2605.11739, 2026. doi: 10.48550/ARXIV.2605.11739. URL https://doi.org/10.48550/arXiv.2605.11739.

Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting. *CoRR*, abs/2510.18874, 2025. doi: 10.48550/ARXIV.2510.18874. URL https://doi.org/10.48550/arXiv.2510.18874.

Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024a. URL http://papers.nips.cc/paper_files/paper/2024/hash/2f8ee6a3d766b426d2618e555b5aeb39-Abstract-Conference.html.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pp. 6621–6642. PMLR / OpenReview.net, 2024b. URL https://proceedings.mlr.press/v235/chen24j.html.

Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025. URL https://proceedings.mlr.press/v267/chu25c.html.

Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, Jiasen Lu, Taira Anderson, Erin Bransom, Kiana Ehsani, Huong Ngo, Yen-Sung Chen, Ajay Patel, Mark Yatskar, Chris Callison-Burch, Andrew Head, Rose Hendrix, Favyen Bastani, Eli VanderBilt, Nathan Lambert, Yvonne Chou, Arnavi Chheda, Jenna Sparks, Sam Skjonsberg, Michael Schmitz, Aaron Sarnat, Byron Bischoff, Pete Walsh, Chris Newell, Piper Wolters, Tanmay Gupta, Kuo-Hao Zeng, Jon Borchardt, Dirk Groeneveld, Crystal Nam, Sophie Lebrecht, Caitlin Wittlif, Carissa Schoenick, Oscar Michel, Ranjay Krishna, Luca Weihs, Noah A. Smith, Hannaneh Hajishirzi, Ross B. Girshick, Ali Farhadi, and Aniruddha Kembhavi. Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*, pp. 91–104. Computer Vision Foundation / IEEE, 2025. doi: 10.1109/CVPR52734.2025.00018. URL https://openaccess.thecvf.com/content/CVPR2025/html/Deitke_Molmo_and_PixMo_Open_Weights_and_Open_Data_for_State-of-the-Art_CVPR_2025_paper.html.

Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, Yunsheng Wu, Rongrong Ji, Caifeng Shan, and Ran He. MME: A comprehensive evaluation benchmark for multimodal large language models. In Danielle Belgrave, Cheng Zhang, Laura N. Montoya, Hsuan-Tien Lin, Razvan Pascanu, Piotr Koniusz, Marzyeh Ghassemi, Nancy Chen, Iván Vladimir Meza Ruíz, and Arturo Loaiza-Bonilla (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025a. URL http://papers.nips.cc/paper_files/paper/

2025/hash/d79a27cf2772fe00be7f341efc0eb517-Abstract-Datasets_and_Benchmarks_Track.html.

Tairan Fu, Miguel González, Javier Conde, Elena Merino-Gómez, and Pedro Reviriego. Have multimodal large language models (mllms) really learned to tell the time on analog clocks? *IEEE Internet Computing*, 2025b.

Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *CoRR*, abs/2603.25562, 2026. doi: 10.48550/ARXIV.2603.25562. URL https://doi.org/10.48550/arXiv.2603.25562.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=Sx038qxjek.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=5h0qf7IBZZ.

Çaglar Gülçehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (rest) for language modeling. *CoRR*, abs/2308.08998, 2023. doi: 10.48550/ARXIV.2308.08998. URL https://doi.org/10.48550/arXiv.2308.08998.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, Hao Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, Tao Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638, 2025. doi: 10.1038/S41586-025-09422-Z. URL https://doi.org/10.1038/s41586-025-09422-z.

Yinghui He, Simran Kaur, Adithya Bhaskar, Yongjin Yang, Jiarui Liu, Narutatsu Ri, Liam Fowl, Abhishek Panigrahi, Danqi Chen, and Sanjeev Arora. Self-distillation zero: Self-revision turns binary rewards into dense supervision. *CoRR*, abs/2604.12002, 2026. doi: 10.48550/ARXIV.2604.12002. URL https://doi.org/10.48550/arXiv.2604.12002.

Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531, 2015. URL http://arxiv.org/abs/1503.02531.

Wenjin Hou, Shangpin Peng, Weinong Wang, Zheng Ruan, Yue Zhang, Zhenglin Zhou, Mingqi Gao, Yifei Chen, Kaiqi Wang, Hongming Yang, Chengquan Zhang, Zhuotao Tian, Han Hu, Yi Yang, Fei Wu, and Hehe Fan. Uni-opd: Unifying on-policy distillation with a dual-perspective recipe. *CoRR*, abs/2605.03677, 2026. doi: 10.48550/ARXIV.2605.03677. URL https://doi.org/10.48550/arXiv.2605.03677.

Maggie Huan, Yuetai Li, Tuney Zheng, Xiaoyu Xu, Seungone Kim, Minxin Du, Radha Poovendran, Graham Neubig, and Xiang Yue. Does math reasoning improve general LLM capabilities? understanding transferability of LLM reasoning. *CoRR*, abs/2507.00432, 2025. doi: 10.48550/ARXIV.2507.00432. URL https://doi.org/10.48550/arXiv.2507.00432.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *CoRR*, abs/2601.20802, 2026. doi: 10.48550/ARXIV.2601.20802. URL https://doi.org/10.48550/arXiv.2601.20802.

Yuxuan Jiang and Francis Ferraro. Bridging reasoning trajectories in on-policy distillation via near-future guidance. *CoRR*, abs/2606.00305, 2026. doi: 10.48550/ARXIV.2606.00305. URL https://doi.org/10.48550/arXiv.2606.00305.

Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Min Joon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling (eds.), *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part IV*, volume 9908 of *Lecture Notes in Computer Science*, pp. 235–251. Springer, 2016. doi: 10.1007/978-3-319-46493-0_15. URL https://doi.org/10.1007/978-3-319-46493-0_15.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In Jian Su, Xavier Carreras, and Kevin Duh (eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, pp. 1317–1327. The Association for Computational Linguistics, 2016. doi: 10.18653/V1/D16-1139. URL https://doi.org/10.18653/v1/d16-1139.

James Kirkpatrick, Razvan Pascanu, Neil C. Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *CoRR*, abs/1612.00796, 2016. URL http://arxiv.org/abs/1612.00796.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as bayesian inference. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, volume EMNLP 2022 of *Findings of ACL*, pp. 1083–1091. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.FINDINGS-EMNLP.77. URL https://doi.org/10.18653/v1/2022.findings-emnlp.77.

Song Lai, Haohan Zhao, Rong Feng, Changyi Ma, Wenzhuo Liu, Hongbo Zhao, Xi Lin, Dong Yi, Min Xie, Qingfu Zhang, Hongbin Liu, Gaofeng Meng, and Fei Zhu. Reinforcement fine-tuning naturally mitigates forgetting in continual post-training. *CoRR*, abs/2507.05386, 2025. doi: 10.48550/ARXIV.2507.05386. URL https://doi.org/10.48550/arXiv.2507.05386.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tülu 3: Pushing frontiers in open language model post-training. *CoRR*, abs/2411.15124, 2024. doi: 10.48550/ARXIV.2411.15124. URL https://doi.org/10.48550/arXiv.2411.15124.

Niklas Lauffer, Xiang Deng, Srivatsa Kundurthy, Brad Kenstler, and Jeff Da. Imitation learning for multi-turn LM agents via on-policy expert corrections. *CoRR*, abs/2512.14895, 2025. doi: 10.48550/ARXIV.2512.14895. URL https://doi.org/10.48550/arXiv.2512.14895.

Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: common objects in context. In David J. Fleet, Tomás Pajdla, Bernt Schiele, and Tinne Tuytelaars (eds.), *Computer Vision - ECCV 2014 - 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V*, volume 8693 of *Lecture Notes in Computer Science*, pp. 740–755. Springer, 2014. doi: 10.1007/978-3-319-10602-1_48. URL https://doi.org/10.1007/978-3-319-10602-1_48.

Wenye Lin and Kai Han. Surgical post-training: Cutting errors, keeping knowledge. *CoRR*, abs/2603.01683, 2026. doi: 10.48550/ARXIV.2603.01683. URL https://doi.org/10.48550/arXiv.2603.01683.

Tao Liu, Taiqiang Wu, Runming Yang, Shaoning Sun, Junjie Wang, and Yujiu Yang. Profit: Leveraging high-value signals in SFT via probability-guided token selection. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics, ACL 2026, San Diego, California, United States, July 2-7, 2026*, pp. 15383–15401. Association for Computational Linguistics, 2026. doi: 10.18653/V1/2026.FINDINGS-ACL.755. URL https://doi.org/10.18653/v1/2026.findings-acl.755.

Kevin Lu and Thinking Machines Lab. On-policy distillation. Thinking Machines Lab: Connectionism, https://thinkingmachines.ai/blog/on-policy-distillation, 2025.

Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/11332b6b6cf4485b84afadb1352d3a9a-Abstract-Conference.html.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *CoRR*, abs/2308.08747, 2023. doi: 10.48550/ARXIV.2308.08747. URL https://doi.org/10.48550/arXiv.2308.08747.

Jiaming Lv, Haoyuan Yang, and Peihua Li. Wasserstein distance rivals kullback-leibler divergence for knowledge distillation. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/78526d7ad4a2532bd91416e948b9644c-Abstract-Conference.html.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022*, volume ACL 2022

of *Findings of ACL*, pp. 2263–2279. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.FINDINGS-ACL.177. URL https://doi.org/10.18653/v1/2022.findings-acl.177.

Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. Docvqa: A dataset for VQA on document images. In *IEEE Winter Conference on Applications of Computer Vision, WACV 2021, Waikoloa, HI, USA, January 3-8, 2021*, pp. 2199–2208. IEEE, 2021. doi: 10.1109/WACV48630.2021.00225. URL https://doi.org/10.1109/WACV48630.2021.00225.

Minesh Mathew, Viraj Bagal, Rubèn Tito, Dimosthenis Karatzas, Ernest Valveny, and C. V. Jawahar. Infographicvqa. In *IEEE/CVF Winter Conference on Applications of Computer Vision, WACV 2022, Waikoloa, HI, USA, January 3-8, 2022*, pp. 2582–2591. IEEE, 2022. doi: 10.1109/WACV51458.2022.00264. URL https://doi.org/10.1109/WACV51458.2022.00264.

nrehiew. SFT, RL, and on-policy distillation through a distributional lens. Blog post, https://nrehiew.github.io/blog/sft_rl_opd/, 2026.

OpenAI. GPT-5.5 system card. https://openai.com/index/gpt-5-5-system-card/, 2026.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=jKHmjlpViu.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html.

Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL http://proceedings.mlr.press/v15/ross11a/ross11a.pdf.

John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In Francis R. Bach and David M. Blei (eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pp. 1889–1897. JMLR.org, 2015. URL http://proceedings.mlr.press/v37/schulman15.html.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL http://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL https://doi.org/10.48550/arXiv.2402.03300.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. Rl's razor: Why online reinforcement learning forgets less. *CoRR*, abs/2509.04259, 2025. doi: 10.48550/ARXIV.2509.04259. URL https://doi.org/10.48550/arXiv.2509.04259.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *CoRR*, abs/2601.19897, 2026. doi: 10.48550/ARXIV.2601.19897. URL https://doi.org/10.48550/arXiv.2601.19897.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by distilling context. *CoRR*, abs/2209.15189, 2022. doi: 10.48550/ARXIV.2209.15189. URL https://doi.org/10.48550/arXiv.2209.15189.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, 2020.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pp. 13484–13508. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.754. URL https://doi.org/10.18653/v1/2023.acl-long.754.

Zifu Wang, Junyi Zhu, Bo Tang, Zhiyu Li, Feiyu Xiong, Jiaqian Yu, and Matthew B. Blaschko. Jigsaw-r1: A study of rule-based visual reinforcement learning with jigsaw puzzles. *Trans. Mach. Learn. Res.*, 2025, 2025. URL https://openreview.net/forum?id=XqQCsuyPve.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=gEZrGCozdqR.

Mingqi Wu, Zhihao Zhang, Qiaole Dong, Zhiheng Xi, Jun Zhao, Senjie Jin, Xiaoran Fan, Yuhao Zhou, Huijie Lv, Ming Zhang, Yanwei Fu, Qin Liu, Songyang Zhang, and Qi Zhang. Reasoning or memorization? unreliable results of reinforcement learning due to data contamination. In Sven Koenig, Chad Jenkins, and Matthew E. Taylor (eds.), *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pp. 33944–33952. AAAI Press, 2026a. doi: 10.1609/AAAI.V40I40.40687. URL https://doi.org/10.1609/aaai.v40i40.40687.

Yangzhen Wu, Shanda Li, Zixin Wen, Xin Zhou, Ameet Talwalkar, Yiming Yang, Wenhao Huang, and Tianle Cai. Learn hard problems during RL with reference guided fine-tuning. *CoRR*, abs/2603.01223, 2026b. doi: 10.48550/ARXIV.2603.01223. URL https://doi.org/10.48550/arXiv.2603.01223.

xAI. Realworldqa: A benchmark for real-world spatial understanding, 2024. URL https://huggingface.co/datasets/xai-org/RealworldQA.

Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative knowledge distillation: Bridging the teacher-student gap through interleaved sampling. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=EgJhwYR2tB.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=N8N0hgNDRt.

Qianhao Yuan, Jie Lou, Xing Yu, Hongyu Lin, Le Sun, Xianpei Han, and Yaojie Lu. Vision-opd: Learning to see fine details for multimodal llms via on-policy self-distillation. *CoRR*, abs/2605.18740, 2026. doi: 10.48550/ARXIV.2605.18740. URL https://doi.org/10.48550/arXiv.2605.18740.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pp. 57905–57923. PMLR / OpenReview.net, 2024. URL https://proceedings.mlr.press/v235/yuan24d.html.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/639a9a172c044fbb64175b5fad42e9a5-Abstract-Conference.html.

Zhihao Zhang, Qiaole Dong, Qi Zhang, Enyu Zhou, Jun Zhao, Zhiheng Xi, Senjie Jin, Xiaoran Fan, Yuhao Zhou, Mingqi Wu, et al. Why reinforcement fine-tuning enables mllms preserve prior knowledge better: A data perspective. In *International Conference on Learning Representations*, volume 2026, pp. 15201–15228, 2026.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *CoRR*, abs/2601.18734, 2026. doi: 10.48550/ARXIV.2601.18734. URL https://doi.org/10.48550/arXiv.2601.18734.

He Zhu, Junyou Su, Peng Lai, Ren Ma, Wenjia Zhang, Linyi Yang, and Guanhua Chen. Anchored supervised fine-tuning. *CoRR*, abs/2509.23753, 2025a. doi: 10.48550/ARXIV.2509.23753. URL https://doi.org/10.48550/arXiv.2509.23753.

Wenhong Zhu, Ruobing Xie, Rui Wang, Xingwu Sun, Di Wang, and Pengfei Liu. Proximal supervised fine-tuning. *CoRR*, abs/2508.17784, 2025b. doi: 10.48550/ARXIV.2508.17784. URL https://doi.org/10.48550/arXiv.2508.17784.

A Derivation of the Idealized Target

Section 3.3 states three results without derivation: that the trust-region problem of Equation 4 is solved by a tilted distribution, that this tilt turns into a hard correctness constraint in the limit of a binary verifier, and that the resulting target costs $1/Z_\theta(x)$ samples to obtain by rejection whereas a revision costs $1/S_\theta(x)$. This appendix derives the three results and, for each one, says what it is used for. Notation follows the main text throughout.

A.1 Setting and Notation

Everything below concerns one prompt x at one point during training.

- y is a complete response and $\mathcal{Y}$ is the set of responses the model can emit; every sum below runs over $\mathcal{Y}$.
- $y^\star$ is the expert reference response paired with x in the demonstration set $\mathcal{D}$.
- $v(y, y^\star) \in \{0, 1\}$ is the verifier, equal to 1 exactly when y solves the task, and $\mathcal{C}(x) = \{y : v(y, y^\star) = 1\}$ is the set of responses it accepts. When the verifier is used as a reward we write $r(y) = v(y, y^\star)$.
- $\pi_\theta(\cdot \mid x)$ is the current policy.

- $\beta > 0$ weights the KL penalty. We denote the corresponding optimum by $\pi^\star_\beta$ and its $\beta \to 0^+$ limit by $\pi^\star$. These are idealized targets, not claims about the policy reached by training.
- $Z_\theta(x) = \Pr_{y\sim\pi_\theta(\cdot|x)}[y \in \mathcal{C}(x)]$ is the acceptance probability of an initial rollout.

A.2 Solving the Trust-Region Problem

Written out, the objective maximized in Equation 4 is

$$J(\pi) = \underbrace{\sum_y \pi(y \mid x)\, r(y)}_{\text{expected verifier reward}} \; - \beta \underbrace{\sum_y \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_\theta(y \mid x)}}_{\mathrm{KL}(\pi(\cdot|x)\,\|\,\pi_\theta(\cdot|x))},$$

maximized over distributions $\pi(\cdot \mid x)$ subject to $\sum_y \pi(y \mid x) = 1$ and $\pi(y \mid x) \geq 0$. Attaching a multiplier λ to the normalization constraint gives the Lagrangian $L(\pi, \lambda) = J(\pi) - \lambda\big(\sum_y \pi(y \mid x) - 1\big)$, whose derivative with respect to the probability of one response is

$$\frac{\partial L}{\partial \pi(y \mid x)} = r(y) - \beta \left[\log \frac{\pi(y \mid x)}{\pi_\theta(y \mid x)} + 1\right] - \lambda.$$

Setting it to zero and solving for $\pi(y \mid x)$ gives

$$\log \frac{\pi(y \mid x)}{\pi_\theta(y \mid x)} = \frac{r(y) - \lambda}{\beta} - 1 \qquad \Longrightarrow \qquad \pi(y \mid x) = \pi_\theta(y \mid x)\, \exp\big(r(y)/\beta\big) \cdot e^{-\lambda/\beta - 1}.$$

The trailing factor is the same for every response, so it is fixed by normalization rather than by solving for λ, and we obtain

$$\pi^\star_\beta(y \mid x) = \frac{\pi_\theta(y \mid x)\, \exp\big(r(y)/\beta\big)}{N_\beta(x)}, \qquad N_\beta(x) = \sum_{y'} \pi_\theta(y' \mid x) \exp\big(r(y')/\beta\big). \tag{10}$$

A.3 The Limit of a Binary Verifier

We now specialize Equation 10 to our reward. Because $r(y) = v(y, y^\star)$ takes only the values 0 and 1, the factor $\exp(r(y)/\beta)$ takes only the value $e^{1/\beta}$ on accepted responses and 1 on rejected ones. The normalizer therefore splits into two sums,

$$N_\beta(x) = \sum_{y \in \mathcal{C}(x)} \pi_\theta(y \mid x)\, e^{1/\beta} + \sum_{y \notin \mathcal{C}(x)} \pi_\theta(y \mid x) = Z_\theta(x)\, e^{1/\beta} + \big(1 - Z_\theta(x)\big),$$

and substituting it back into Equation 10 gives

$$\pi^\star_\beta(y \mid x) = \begin{cases} \dfrac{\pi_\theta(y \mid x)}{Z_\theta(x) + \big(1 - Z_\theta(x)\big)e^{-1/\beta}}, & y \in \mathcal{C}(x), \\[2ex] \dfrac{\pi_\theta(y \mid x)}{Z_\theta(x)\, e^{1/\beta} + \big(1 - Z_\theta(x)\big)}, & y \notin \mathcal{C}(x), \end{cases} \tag{11}$$

where the first line follows after dividing numerator and denominator by $e^{1/\beta}$.

Before taking a limit it helps to see what β actually controls here. In the objective β is the weight of the proximity penalty, but in the solution it enters only through the factor $e^{1/\beta}$, which is the advantage that being accepted confers. Comparing an accepted y with a rejected y', and summing the second line of Equation 11 over all rejected responses,

$$\frac{\pi^\star_\beta(y \mid x)}{\pi^\star_\beta(y' \mid x)} = \frac{\pi_\theta(y \mid x)}{\pi_\theta(y' \mid x)} \cdot e^{1/\beta}, \qquad \sum_{y \notin \mathcal{C}(x)} \pi^\star_\beta(y \mid x) = \frac{1 - Z_\theta(x)}{Z_\theta(x)\, e^{1/\beta} + 1 - Z_\theta(x)}.$$

A large β makes $e^{1/\beta} \approx 1$, so the optimum stays close to $\pi_\theta(\cdot \mid x)$ and leaves nearly the original mass $1 - Z_\theta(x)$ on responses the verifier rejects; shrinking β raises $e^{1/\beta}$ and drives that mass towards zero.

Correctness is thus only ever a soft preference: at any finite β the optimum is a mixture that still places probability on incorrect responses.

Taking $\beta \to 0^+$ makes $1/\beta \to \infty$. In the first line of Equation 11 the term $e^{-1/\beta}$ vanishes and the denominator tends to $Z_\theta(x)$, while in the second line $e^{1/\beta}$ diverges and the whole expression tends to zero. Provided $Z_\theta(x) > 0$,

$$\pi^\star(y \mid x) = \lim_{\beta \to 0^+} \pi^\star_\beta(y \mid x) = \frac{\pi_\theta(y \mid x)\, \mathbf{1}\{y \in \mathcal{C}(x)\}}{Z_\theta(x)}, \tag{12}$$

which is Equation 5.

Equation 12 is the conditional law of $y \sim \pi_\theta(\cdot \mid x)$ given $y \in \mathcal{C}(x)$: correct responses keep the relative probabilities the current policy gave them and incorrect ones are dropped. Proximity therefore survives the limit, since the target is not uniform on $\mathcal{C}(x)$ but still weighted by $\pi_\theta(y \mid x)$, which is

A.4 Where Each Objective Sits

Equation 12 has exactly two ingredients: a proposal, $\pi_\theta(\cdot \mid x)$, and a verified indicator $\mathbf{1}\{y \in \mathcal{C}(x)\}$ normalized by $Z_\theta(x)$. Each objective of Section 3.2 changes one of them, and reading off which one explains both the failures and the fix.

Rejection sampling and RLVR change neither. They draw $\hat{y}_1, \hat{y}_2, \ldots$ from $\pi_\theta(\cdot \mid x)$ and keep the first accepted draw, which is an unbiased sample from Equation 12. The index N of that draw is geometric with success probability $Z_\theta(x)$, so

$$\mathbb{E}[N] = \sum_{k \geq 1} k \left(1 - Z_\theta(x)\right)^{k-1} Z_\theta(x) = \frac{1}{Z_\theta(x)},$$

and no target is ever produced when $Z_\theta(x) = 0$, the limit the cold-start tasks approach with Z_θ below 1% (Appendix C.1).

SFT keeps the indicator and discards the π_θ factor.

We identify the distribution that SFT actually fits. Write $\delta_{y^\star}$ for the point mass at the reference, that is $\delta_{y^\star}(y) = 1$ for $y = y^\star$ and 0 for every other response, which is the objective of Equation 2. The target SFT fits is therefore $\delta_{y^\star}$.

We compare $\delta_{y^\star}$ with Equation 12. Both put all their mass inside $\mathcal{C}(x)$, so both satisfy the indicator. They differ in how that mass is distributed: $\pi^\star$ spreads it over every accepted response in proportion to $\pi_\theta(y \mid x)$, whereas $\delta_{y^\star}$ puts all of it on one accepted response chosen without consulting π_θ at all. This is the precise sense in which SFT replaces $\pi^\star$ by $y^\star$: the indicator is kept, the π_θ factor is dropped. The disagreement is quantitative as well. Equation 12 assigns the reference probability $\pi^\star(y^\star \mid x) = \pi_\theta(y^\star \mid x)/Z_\theta(x)$ while $\delta_{y^\star}$ assigns it 1, so SFT over-weights the reference by

$$\frac{1}{\pi^\star(y^\star \mid x)} = \frac{Z_\theta(x)}{\pi_\theta(y^\star \mid x)} \;\geq\; 1,$$

since $Z_\theta(x) = \sum_{y \in \mathcal{C}(x)} \pi_\theta(y \mid x)$ includes the term $\pi_\theta(y^\star \mid x)$; the two coincide only when the current policy gives no probability to any other accepted response. The factor grows without bound as $y^\star$ is not sampled from $\pi_\theta(\cdot \mid x)$, which is the cold-start regime, and one quantity therefore drives both of SFT's observed behaviors: an explicit target that works from near-zero accuracy, and an update large enough to forget.

Self-distillation keeps the proposal and replaces the indicator. It targets the same $\pi^\star$, but does not use the verifier during training, so it cannot use the characterization in Equation 12 and instead puts the reference-conditioned model in its place; dividing that model by $\pi_\theta(y \mid x)$ then writes it as a tilt of the same form as Equation 10,

$$\pi^\star(y \mid x) \;\approx\; \pi_\theta(y \mid x, y^\star) \;=\; \pi_\theta(y \mid x)\, \exp\big(\Delta_\theta(y)\big), \qquad \Delta_\theta(y) = \log \frac{\pi_\theta(y \mid x, y^\star)}{\pi_\theta(y \mid x)}, \tag{13}$$

Comparing Equation 13 with Equation 12, the hard indicator has become the soft exponent Δ_θ, which no verifier constrains. Two consequences follow, and they are the two defects the main text reports:

the target need not place all its mass on $\mathcal{C}(x)$, so correctness is not guaranteed, and the exponent is carried by per-token terms a_n that need not sit at the positions deciding acceptance, which is what η measures.

ReDraft keeps the indicator and replaces the proposal. A candidate is produced in two steps: a rollout $\hat{y} \sim \pi_\theta(\cdot \mid x)$, followed by a revision $\tilde{y} \sim R_\theta(\cdot \mid x, y^\star, \hat{y})$, where R_θ is the distribution of the response the model emits when given the prompt, the reference and its own rollout, and returns $\hat{y}$ unchanged when that rollout is already accepted. Only the candidate is used for training, so what we need is the law of $\tilde{y}$ on its own, obtained by averaging over the intermediate rollout:

$$\rho_\theta(y \mid x) = \sum_{\hat{y}} \pi_\theta(\hat{y} \mid x)\, R_\theta(y \mid x, y^\star, \hat{y})\,. \tag{14}$$

Here $\rho_\theta(\cdot \mid x)$ is the distribution of responses produced by "draft an answer, then edit it against the reference", in the same sense that $\pi_\theta(\cdot \mid x)$ is the distribution produced by "answer directly". It takes the place of $\pi_\theta(\cdot \mid x)$ as the proposal, and nothing else about the procedure changes.

The test itself is unchanged: $\mathcal{C}(x)$ is the same set of verifier-accepted responses as before, independent of how a candidate was produced, so a candidate is retained if and only if $\tilde{y} \in \mathcal{C}(x)$. Under the new proposal that happens with probability

$$S_\theta(x) = \Pr_{\tilde{y} \sim \rho_\theta(\cdot \mid x)}[\tilde{y} \in \mathcal{C}(x)] = \sum_{y \in \mathcal{C}(x)} \rho_\theta(y \mid x),$$

the same sum that defines $Z_\theta(x)$ over the same set, but taken against ρ_θ instead of π_θ.

A training target is the first candidate to pass the test, so its distribution follows from summing over the number of candidates rejected before it. Candidates are drawn independently, so the k-th of them is the first to be retained and equals y when the first $k-1$ draws are rejected, of probability $\big(1 - S_\theta(x)\big)^{k-1}$, and the k-th draw is y and accepted, of probability $\rho_\theta(y \mid x)\mathbf{1}\{y \in \mathcal{C}(x)\}$. Summing over k with $\sum_{k \geq 1}(1-p)^{k-1} = 1/p$,

$$\pi_{\text{ReDraft}}(y \mid x) = \sum_{k \geq 1} \big(1 - S_\theta(x)\big)^{k-1} \rho_\theta(y \mid x)\, \mathbf{1}\{y \in \mathcal{C}(x)\} = \frac{\rho_\theta(y \mid x)\, \mathbf{1}\{y \in \mathcal{C}(x)\}}{S_\theta(x)}. \tag{15}$$

Discarding the rejected candidates has therefore sampled ρ_θ conditioned on acceptance: Equation 15 is Equation 12 with (π_θ, Z) replaced by (ρ_θ, S_θ), derived rather than assumed. The indicator survives, so every retained target is correct, and the expected number of attempts per target is $1/S_\theta(x)$.

The two acceptance rates are related exactly, which is where the cold-start gain comes from. Splitting on whether the rollout already passes, and using that R_θ returns $\hat{y}$ unchanged in that case, so that $\Pr[\tilde{y} \in \mathcal{C}(x) \mid \hat{y} \in \mathcal{C}(x)] = 1$,

$$S_\theta(x) = \underbrace{\Pr\big[\hat{y} \in \mathcal{C}(x)\big]}_{Z_\theta(x)} \cdot 1 + \underbrace{\Pr\big[\hat{y} \notin \mathcal{C}(x)\big]}_{1 - Z_\theta(x)} \cdot \underbrace{\Pr\big[\tilde{y} \in \mathcal{C}(x) \,\big|\, \hat{y} \notin \mathcal{C}(x)\big]}_{r_\theta(x)},$$

which is the identity in Equation 8. Revision keeps the $Z_\theta(x)$ that rejection sampling already collects and adds the term $\big(1 - Z_\theta(x)\big) r_\theta(x)$ recovered from the failures that rejection sampling discards. At cold start the first term is negligible and the second is the entire yield, so $S_\theta(x) > 0$ is possible exactly where $Z_\theta(x) = 0$: the events differ because $\tilde{y}$ is drawn with the reference in context, from a distribution whose support is not that of $\pi_\theta(\cdot \mid x)$.

The substitution is not free: $\rho_\theta \neq \pi_\theta$, so Equation 15 is a biased surrogate for Equation 12. The minimal-edit instruction biases $\tilde{y}$ towards $\hat{y}$ but does not constrain it, and a repair is sometimes a full rewrite, so we do not bound the discrepancy analytically. What holds is that $\tilde{y}$ is emitted by the policy itself, and Section 6 measures the discrepancy that remains as target perplexity under the generating model.

B Experiment Setup Details

B.1 Dataset Construction

Each target task draws on an existing image corpus: for Counting we sample 2k images from PixMo-Count (Deitke et al., 2025); for Clock Reading we take 10k standard clock images from

Analog Clocks Combinations (Fu et al., 2025b); and for Jigsaw we sample 10k 640×480 images from COCO (Lin et al., 2014), cut each into four equal patches, and shuffle them at random. Every prompt is then paired with an expert response $y^\star$ generated by GPT-5.5 (OpenAI, 2026), and the same responses serve every objective that needs one: they are the SFT targets, the reference ReDraft revises against, and the response on which OPSD conditions its teacher.

B.2 Hyper-parameters

We implement all experiments on top of the TRL framework (von Werra et al., 2020). Across all runs, we use a learning rate of 1×10^{-5} and a batch size of 32. We set `max_length` to either 4,096 or 2,048, depending on the length of the training corpus. For the OPSD experiments, we set `token_clip` to 0.0, which disables element-wise clipping and therefore optimizes the exact forward KL, and set the temperature to 1.1. At evaluation time on the target tasks, we decode with a temperature of 0.7 and sample 8 responses per question, reporting the $avg@8$.

C Per-Task Acquisition and Retention Results

C.1 Sampling and Repair Rates

Assumption 1 requires $r_\theta(x) > 0$, and Equation 8 makes ReDraft's supply of training targets depend on it. We measure both rates on the training corpora by sampling one rollout per prompt and, when the verifier rejects it, revising it once with the reference in context (Table 3).

The cold-start tasks show the gap the identity predicts. At 3B the model solves 0.21% of Jigsaw and 0.03% of Clock Reading prompts unaided, so rejection sampling collects almost nothing, but 92.25% and 56.84% of the failed rollouts are repairable, giving targets for 92.27% and 56.85% of prompts. On Counting, where sampling already works, revision still raises the yield from 65.85% to 92.00%.

Table 3: Sampling success, repair rate, and target yield. N is the number of prompts in the corpus, Z_θ the fraction solved by the first rollout, r_θ the fraction of the remaining prompts whose revision the verifier accepts, and $S_\theta = Z_\theta + (1 - Z_\theta)r_\theta$ the fraction that contributes a training target.

Task	Model	N	Sampling (Z_θ)	Repair (r_θ)	Yield (S_θ)
Counting	3B	2,000	65.85	76.57	92.0
Jigsaw	3B	10,000	0.21	92.25	92.27
Clock Reading	3B	10,000	0.03	56.84	56.85
Counting	7B	2,000	77.40	98.23	99.6
Jigsaw	7B	10,000	6.09	94.0	94.37
Clock Reading	7B	10,000	0.67	64.97	65.2

C.2 RLVR at Cold Start

Section 3.3 attributes RLVR's weakness at cold start to the absence of positive rollouts rather than to insufficient training. We test this by running GRPO on the two cold-start tasks at both scales, sampling $G = 8$ rollouts for each of 32 prompts per step at a learning rate of 2×10^{-6} and dropping groups whose rollouts all receive the same reward.

Figure 11 separates a rising reward from an acquired skill: the format term saturates everywhere while the outcome term moves only in a single setting. Clock Reading never leaves zero at either scale, and Jigsaw at 3B sits at the accuracy of guessing an ordering at random, so neither is learning. Only Jigsaw at 7B improves, and two properties of that setting supply what the others lack: its answer space is small enough that a correct ordering is occasionally sampled by chance, and the 7B model already solves 6.09% of these prompts unaided against 0.21% at 3B (Table 3). Verifiable rewards therefore presuppose some of the competence they are meant to build, whereas revision reaches a 56.85% yield on the same 3B Clock Reading corpus on which GRPO obtains nothing.

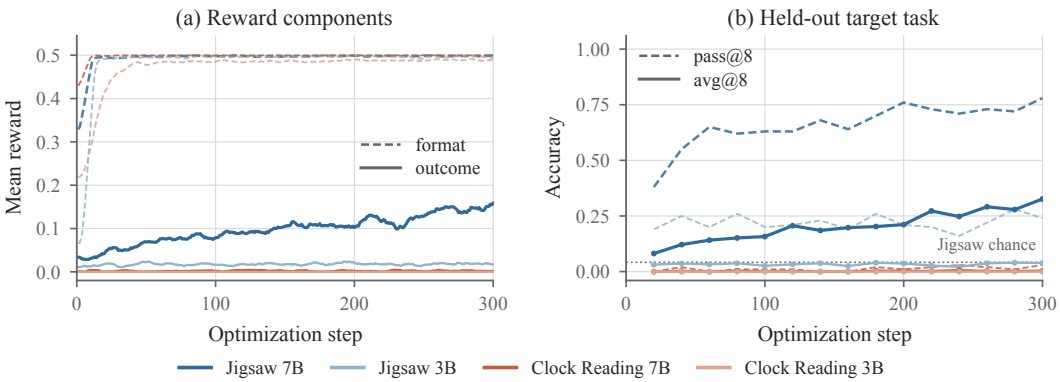


Figure 11: GRPO on the two cold-start tasks. (a) The two reward components during training. The format term reaches it in every setting, while the outcome term rises only for Jigsaw at 7B. (b) Accuracy on 100 held-out prompts, evaluated with 8 samples per prompt. The dotted line marks the accuracy of guessing a 2×2 ordering at random.

C.3 Per-Setting Results

Table 4 expands the aggregate comparison of Section 5.1 into individual runs. Each target task is trained separately on Qwen2.5-VL-3B and 7B, and every row is one task at its selected checkpoint: Target and Prior are its target-task and prior-capability scores, and Gain and Loss are the corresponding changes from the base model. Reject Sampling is evaluated only on Counting, where the base policy already has substantial competence. The means over these rows are the values reported in the main text and plotted in Figure 4.

Table 4: Per-setting results. Target and Prior are its scores, while Gain is target improvement and Loss is the signed change in prior-task performance relative to the corresponding base model, all in percentage points. ReDraft matches or exceeds SFT's target score, sometimes over more steps, and retains more prior-task performance in every setting. Reject Sampling is available only for Counting.

Task	Model	Method	Steps	Target	Prior	Gain ↑	Loss
Counting	3B	SFT	63	54.13	68.91	+10.13	−6.58
		ReDraft	104	56.75	76.00	+12.75	+0.50
		Reject Sampling	42	45.00	76.24	+1.00	+0.75
		OPSD	63	51.38	75.16	+7.38	−0.33
Jigsaw	3B	SFT	313	72.50	72.86	+68.75	−2.64
		ReDraft	612	81.25	74.37	+77.50	−1.12
		OPSD	626	6.12	75.34	+2.37	−0.15
Clock Reading	3B	SFT	313	76.38	71.83	+76.38	−3.67
		ReDraft	356	84.88	74.69	+84.88	−0.80
		OPSD	626	43.13	75.02	+43.13	−0.48
Counting	7B	SFT	126	64.38	18.92	+12.13	−61.87
		ReDraft	124	63.62	80.16	+11.37	−0.63
		Reject Sampling	49	53.50	79.89	+1.25	−0.90
		OPSD	126	56.63	60.06	+4.38	−20.73
Jigsaw	7B	SFT	313	75.50	73.46	+70.25	−7.34
		ReDraft	596	76.88	78.75	+71.63	−2.04
		OPSD	626	19.50	78.24	+14.25	−2.56
Clock Reading	7B	SFT	313	80.00	63.11	+79.87	−17.68
		ReDraft	408	83.63	76.02	+83.50	−4.77
		OPSD	626	44.50	67.67	+44.37	−13.12

C.4 Extended OPSD Training

Table 5 breaks Table 2 down by task and model scale. The longer budget leaves acquisition essentially unchanged in most cells: on 3B, the gain moves by 1.6 points on Jigsaw and 0.5 on Clock Reading even though the number of updates grows by 50 to 100%, and Counting stays below 9 points at either scale. The single substantial improvement is 7B Clock Reading, which rises from +44.4 to +61.4

and accounts for most of the difference between the two budgets. Even there OPSD ends 22.1 points below the +83.5 that ReDraft reaches on the same run.

Table 5: OPSD under a longer step budget, per task and model scale. Steps is cumulative, and Gain is target-accuracy improvement in percentage points over base accuracies of 44.00/3.75/0.00 on 3B and 52.25/5.25/0.13 on 7B for Counting, Jigsaw, and Clock Reading. Averaging the two scales column-wise reproduces Table 2.

	Qwen2.5-VL-3B				Qwen2.5-VL-7B			
	standard		extended		standard		extended	
Task	Steps	Gain	Steps	Gain	Steps	Gain	Steps	Gain
Counting	63	+7.38	252	+8.12	126	+4.38	252	+8.13
Jigsaw	626	+2.37	939	+4.00	626	+14.25	939	+15.12
Clock Reading	626	+43.13	1252	+43.63	626	+44.37	939	+61.37
Mean	438	+17.63	814	+18.58	459	+21.00	710	+28.21

D Per-Task Answer-Perplexity Distributions

Figure 12 separates the pooled result by task and model scale. Revision lowers the median in every comparison, and on Counting and Clock Reading it brings the target down to the model's own rollout at both scales: the revised medians are 1.59 and 1.44 on 3B and 1.32 and 1.41 on 7B, against own-rollout values of 1.51, 1.33, 1.23, and 1.26, starting from expert targets between 4.19 and 6.27. Jigsaw is the harder case. At 7B the revised median falls from 7.93 to 2.75, close to the 2.06 of the model's own rollout, but at 3B it stops at 4.00 against an own-rollout 1.28, so the revision there stays some way from the model's distribution even though it is verified correct. The consistently low PPL of incorrect rollouts again separates policy proximity from correctness: the advantage of self-revision is not merely low PPL, but a verified-correct target that remains closer to the model distribution than the SFT trace.

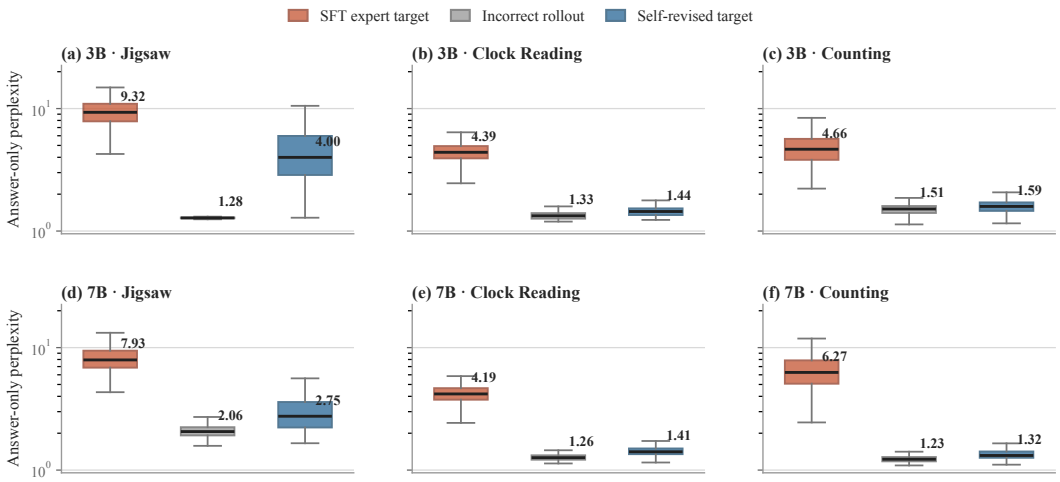


Figure 12: Per-task answer-only PPL distributions. Rows correspond to Qwen2.5-VL-3B/7B and columns to Jigsaw, Clock Reading, and Counting. Each panel contains 400 paired prompts per response source. Boxes span the interquartile range, center lines and labels denote medians, and whiskers extend to 1.5× IQR.

E Additional Weight-Space Results

E.1 Weight-Space Summary

Table 6 collects the weight-space measurements of Section 6. Three properties of the update are measured. Its direction, the cosine against the corresponding SFT update, is closer to SFT for

Table 6: Weight-space signature and task outcomes. Weight statistics are means over all task–model pairs; displacements are reported in units of 10^{-3} and cosines against the corresponding SFT update. The concentration measures are defined in Appendix E.4. Outcomes follow the Table 4.

	SFT	ReDraft (ours)	OPSD
Update magnitude ($\times 10^{-3}$)			
All parameters	6.52	5.59	4.13
Vision encoder	9.07	8.20	8.47
Projector	7.90	6.90	7.00
LM linear layers	7.41	6.43	4.60
Output head	14.27	10.71	3.70
Spectral geometry			
Aggregate effective rank	1397.6	1454.7	1506.4
Top-1% norm share (%)	59.0	54.6	53.7
Leading-direction share $\sigma_1/\|\Delta W\|_F$	0.289	0.258	0.251
Update direction, $\cos(\Delta, \Delta_{\text{SFT}})$			
All parameters	—	$+0.272$	$+0.135$
LM linear layers	—	$+0.237$	$+0.134$
LM vocabulary	—	$\mathbf{+0.764}$	$+0.390$
Task outcomes			
Target-task gain ↑	$+52.9$	$\mathbf{+56.9}$	$+19.3$
Original-task loss ↓	-16.6	$\mathbf{-1.5}$	-6.2

ReDraft than for OPSD, most clearly in the vocabulary tensors (Appendix E.2). Its magnitude, the relative displacement of each module from its pre-trained value, separates the objectives as SFT $>$ ReDraft $>$ OPSD throughout the language model and most sharply at the output head, while the vision encoder and the projector barely differ between objectives (Appendix E.3). Its concentration, the number of singular directions that carry the update, again places ReDraft between the two baselines (Appendix E.4).

E.2 Update Direction

Table 7 gives the cosines behind Figure 8, comparing each objective with SFT for every task and model scale. The full-parameter SFT–ReDraft cosine exceeds SFT–OPSD in every pair, and the vocabulary comparison is likewise unanimous (0.644–0.961 vs. 0.314–0.492).

Table 7: Detailed update-direction agreement. Entries are $\cos(\Delta_A, \Delta_B)$ from the same initialization. "LM linear" is the unweighted mean over attention QKV/output and MLP up–gate/down groups; vocabulary denotes `LLM.embed`.

Task	Model	Pair	All	LM linear	Vocabulary
Counting	3B	SFT–ReDraft	$+0.224$	$+0.250$	$+0.702$
		SFT–OPSD	$+0.148$	$+0.182$	$+0.356$
		ReDraft–OPSD	$+0.079$	$+0.096$	$+0.353$
Clock Reading	3B	SFT–ReDraft	$+0.134$	$+0.079$	$+0.644$
		SFT–OPSD	$+0.120$	$+0.095$	$+0.344$
		ReDraft–OPSD	$+0.098$	$+0.081$	$+0.254$
Jigsaw	3B	SFT–ReDraft	$+0.589$	$+0.571$	$+0.961$
		SFT–OPSD	$+0.121$	$+0.114$	$+0.439$
		ReDraft–OPSD	$+0.122$	$+0.114$	$+0.448$
Counting	7B	SFT–ReDraft	$+0.231$	$+0.241$	$+0.660$
		SFT–OPSD	$+0.182$	$+0.201$	$+0.392$
		ReDraft–OPSD	$+0.117$	$+0.132$	$+0.400$
Clock Reading	7B	SFT–ReDraft	$+0.194$	$+0.102$	$+0.781$
		SFT–OPSD	$+0.141$	$+0.120$	$+0.492$
		ReDraft–OPSD	$+0.124$	$+0.106$	$+0.470$
Jigsaw	7B	SFT–ReDraft	$+0.261$	$+0.177$	$+0.832$
		SFT–OPSD	$+0.097$	$+0.093$	$+0.314$
		ReDraft–OPSD	$+0.085$	$+0.084$	$+0.309$

E.3 Update Magnitude

ReDraft occupies the intermediate update regime. We measure how far each objective moves the weights. For every tensor we take the relative displacement $\|\Delta W\|_F / \|W_0\|_F$ from its pre-trained value, aggregate it by module, and average over the three tasks and both model scales. Figure 13 shows the resulting profile by depth: the objectives separate throughout the language model in the order SFT $>$ ReDraft $>$ OPSD, while the vision-encoder curves nearly coincide. Thus ReDraft changes the weights less than SFT but more than OPSD. Aggregate displacements are $(6.52, 5.59, 4.13) \times 10^{-3}$ for SFT, ReDraft, and OPSD, respectively.

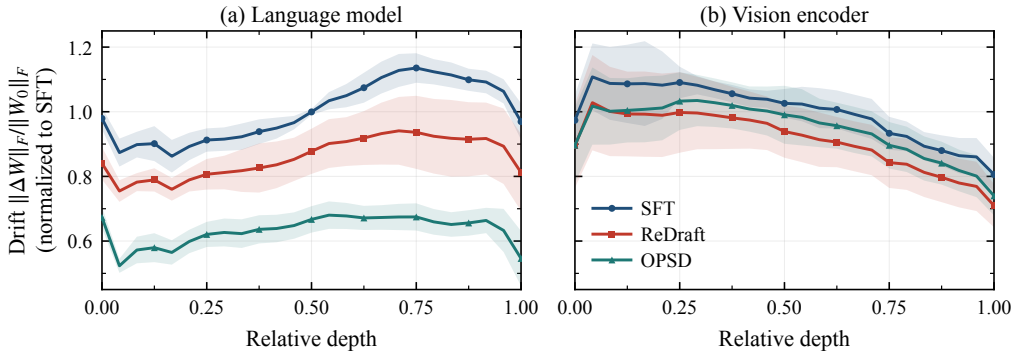


Figure 13: Update magnitude by network depth. Each curve is normalized by the layer-mean drift so that the two model scales can be pooled; curves are means over all task–model pairs.

Table 8 breaks the displacements averaged in Table 6 down by task and model scale, one row per objective and module group. The aggregate ordering of Figure 13 holds separately in each of them: total displacement, LM-linear drift, and output-head drift always satisfy SFT $>$ ReDraft $>$ OPSD. The gap is smallest on Jigsaw, where ReDraft nearly matches SFT, and larger on Counting and Clock Reading.

Table 8: Detailed update magnitude ($\times 10^{-3}$). "All" covers all measured parameters; vision encoder, projector, and LM-linear values are energy-aggregated over matrix weights, while output head denotes `lm_head.weight`. The task–model pairs listed here are the observations averaged in Table 6.

Task	Model	Method	All	Vision enc.	Projector	LM linear	Output head
Counting	3B	SFT	3.862	5.794	4.895	4.138	3.945
		ReDraft	3.261	5.352	4.184	3.440	2.910
		OPSD	2.856	5.621	4.053	2.860	1.564
Clock Reading	3B	SFT	6.882	10.464	8.676	7.063	9.876
		ReDraft	5.050	7.477	6.760	5.396	5.486
		OPSD	4.511	8.950	8.950	4.421	2.897
Jigsaw	3B	SFT	8.257	11.624	9.756	7.853	17.114
		ReDraft	8.033	11.058	9.149	7.629	16.984
		OPSD	5.215	11.097	8.093	4.893	4.369
Counting	7B	SFT	4.070	5.679	4.745	5.467	5.772
		ReDraft	3.365	5.510	4.457	4.440	4.408
		OPSD	2.869	5.928	4.436	3.658	2.475
Clock Reading	7B	SFT	7.293	8.764	8.761	9.160	22.151
		ReDraft	5.836	8.528	7.358	7.507	13.653
		OPSD	4.327	8.595	8.229	5.514	4.870
Jigsaw	7B	SFT	8.755	12.114	10.587	10.808	26.785
		ReDraft	8.003	11.265	9.481	10.169	20.834
		OPSD	5.004	10.633	8.255	6.267	5.995

E.4 Update Concentration

The update is concentrated rather than diffuse. A larger update need not be a more scattered one. Recent work counts the directions an update uses by the effective rank of the update matrix, and reports that on-policy distillation uses fewer of them than RL (Cai et al., 2026). Measured over the language model's attention and MLP matrices, ReDraft is once more intermediate: its effective rank

is 1454.7, below OPSD's 1506.4 and above SFT's 1397.6. Its larger update is therefore packed into fewer directions than OPSD's rather than spread more widely.

The concentration measures are computed from the singular values $\sigma_1 \geq \cdots \geq \sigma_r$ of the update $\Delta W = W - W_0$ of each Attention and MLP weight matrix of the language model, using the full spectrum ($r = \min(m, n)$, no truncation) and averaging over those matrices. Writing $\bar{\sigma}_i = \sigma_i / \sum_j \sigma_j$ for the normalized spectrum, the effective rank is $\exp(-\sum_i \bar{\sigma}_i \log \bar{\sigma}_i)$, the exponential of its entropy, and is lower when the update is carried by fewer directions. The top-k norm share is $\left(\sum_{i \leq k} \sigma_i^2 / \sum_j \sigma_j^2\right)^{1/2}$ and is higher when it is carried by fewer directions; we report it at $k = \lceil r/100 \rceil$, the top 1% of directions, and at $k = 1$, where it reduces to $\sigma_1 / \|\Delta W\|_F$, the fraction of the update carried by its single leading direction. All three are invariant to rescaling ΔW, so they describe the shape of the update rather than its size.

Table 9 reports the three measures for each task. All three place SFT as the most concentrated objective and ReDraft as more concentrated than OPSD.

Table 9: Detailed spectral geometry. Norm shares are reported as percentages. Effective rank is bounded by the matrix dimensions, which differ between scales, so it should be compared between methods within a row rather than across scales.

Task	Model	Method	Effective rank	Top-1% share (%)	$\sigma_1 / \|\Delta W\|_F$
Counting	3B	SFT	1114.0	57.2	0.299
		ReDraft	1142.6	53.6	0.273
		OPSD	1162.6	53.0	0.266
Clock Reading	3B	SFT	958.1	58.3	0.300
		ReDraft	1040.1	53.9	0.278
		OPSD	1077.9	52.0	0.256
Jigsaw	3B	SFT	1032.9	53.5	0.270
		ReDraft	1043.4	52.7	0.264
		OPSD	1112.3	48.5	0.235
Counting	7B	SFT	1879.4	68.3	0.359
		ReDraft	1951.2	57.1	0.258
		OPSD	1982.1	58.5	0.277
Clock Reading	7B	SFT	1602.4	62.2	0.272
		ReDraft	1682.7	60.0	0.267
		OPSD	1781.8	60.3	0.265
Jigsaw	7B	SFT	1799.1	54.2	0.232
		ReDraft	1868.2	50.5	0.209
		OPSD	1921.9	49.7	0.205

F WHERE OPSD PLACES ITS TOKEN-LEVEL CREDIT

OPSD's token credit misses the decisive tokens. Section 3.3 explains why OPSD's token-level supervision need not focus on task-critical errors. Figure 14 examines this issue on two rollouts, colouring each token by $\mathrm{adv}_n = \log p_T(\hat{y}_n) - \log p_S(\hat{y}_n)$, so that blue marks tokens the teacher suppresses and red ones it reinforces. The answer digits that alone decide the task carry little of the weight: most of it is spread over incidental reasoning, style, and formatting tokens, and some answer tokens are even pushed the wrong way. OPSD can thus know the correction and still fail to put its signal where the decision is made, whereas ReDraft writes the correction out as a verified target.

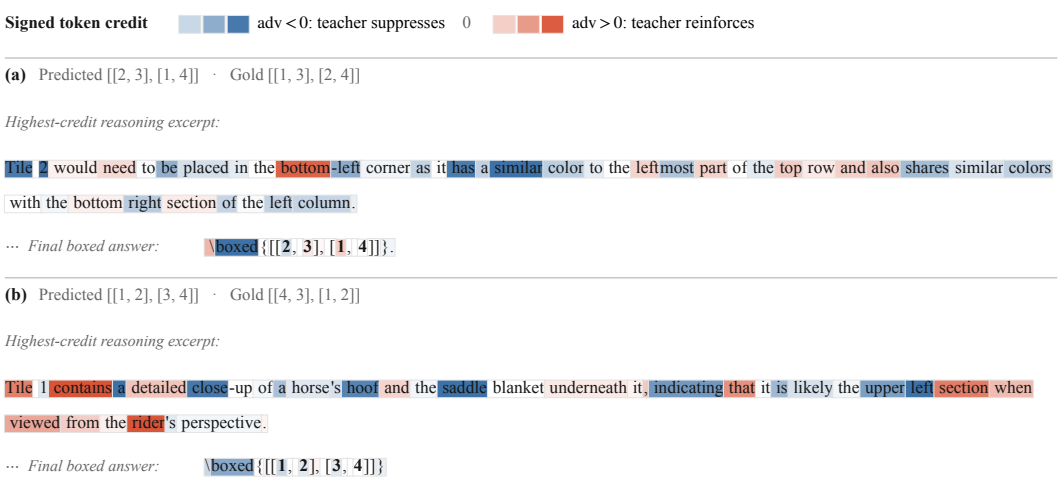


Figure 14: OPSD credit peaks away from the task-deciding answer. Token backgrounds show signed sampled-token credit: blue denotes $\mathrm{adv} < 0$ (teacher suppression), red $\mathrm{adv} > 0$ (teacher reinforcement), and darker color indicates larger $|\mathrm{adv}|$ under one shared scale.

G CASE STUDY

Below we show one self-revision example for each target task. For every example we display the visual input, the problem with its ground-truth answer, the model's incorrect unaided rollout, and the verifier-accepted revised response used as the ReDraft training target.

G.1 CLOCK READING

Figure 15 shows a Clock Reading example. The unaided rollout misreads all three hands and reports `06:08:30`. Given the expert reference only as context for revision, the same model corrects the reading to the verified answer `03:30:38`, which is revised minimally.

Input Image

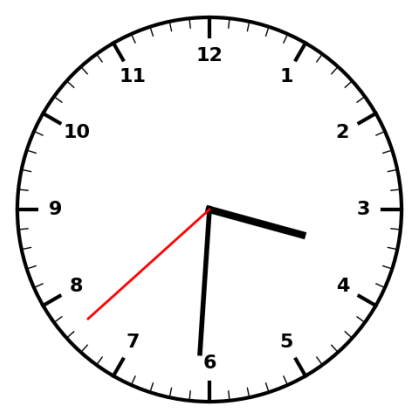


Figure 15: A demonstration of the Clock Reading task.

Example From Clock Reading

Problem:

```
You are given an image of an analogue clock.  The second hand is red
and thin, the minute hand is black and long, and the hour hand is
black, shorter and thicker.

Please answer the following clock-reading question:  determine the
time displayed on the clock face by locating the hour hand, the
minute hand and the second hand, show your detailed step-by-step
reasoning process, and wrap the final time in \boxed{} using the
zero-padded 12-hour HH:MM:SS format (for example, if the clock shows
3:45:20, end with \boxed{03:45:20}).

The question is as follows:

What time is shown on the clock in the given image?

Now, give your detailed reasoning process in no more than 200 words.
```

Answer: `03:30:38`

Incorrect Rollout

```
To determine the time shown on the clock in the given image, we need
to analyze the positions of the hour hand, the minute hand, and the
second hand.
```

```
1.  Hour Hand Analysis:
   - The hour hand is black and longer.
   - It points between the numbers 6 and 7.
   - Counting from 6 to 7, it is approximately halfway between these
two numbers.
2.  Minute Hand Analysis:
   - The minute hand is black and longer.
   - It points between the numbers 8 and 9.
   - Counting from 8 to 9, it is approximately halfway between these
two numbers.
3.  Second Hand Analysis:
   - The second hand is red and thin.
   - It points between the numbers 10 and 11.
   - Counting from 10 to 11, it is approximately halfway between these
two numbers.
4.  Time Calculation:
   - The hour hand is at 6 and 30 seconds (halfway between 6 and 7).
   - The minute hand is at 8 and 30 seconds (halfway between 8 and 9).
   - The second hand is at 10 and 30 seconds (halfway between 10 and
11).
Therefore, the time shown on the clock is \boxed{06:08:30}.
```

Revised Response

```
To determine the time shown on the clock, we need to analyze the
positions of the hour hand, the minute hand, and the second hand.
1.  Hour Hand Analysis:
   - The hour hand is black and shorter.
   - It points between the numbers 3 and 4.
   - Counting from 3 to 4, it is approximately halfway between these
two numbers.
2.  Minute Hand Analysis:
   - The minute hand is black and longer.
   - It points between the numbers 6 and 7.
   - Counting from 6 to 7, it is approximately halfway between these
two numbers.
3.  Second Hand Analysis:
   - The second hand is red and thin.
   - It points between the numbers 7 and 8.
   - Counting from 7 to 8, it is approximately halfway between these
two numbers.
4.  Time Calculation:
   - The hour hand is at 3 and 30 seconds (halfway between 3 and 4).
   - The minute hand is at 6 and 30 seconds (halfway between 6 and 7).
   - The second hand is at 7 and 30 seconds (halfway between 7 and 8).
Therefore, the time shown on the clock is \boxed{03:30:38}.
```

G.2 COUNTING

Figure 16 shows a Counting example. The unaided rollout under-counts people in the photograph (5 instead of 6). After revision minimally against the expert reference, the model adds the missing person and reports the verified count 6, which is revised minimally.

Input Image



Figure 16: A demonstration of the Counting task.

Example From Counting

Problem:

```
You are given an image.  Please answer the following counting
question:  count how many target objects are in the image, show your
detailed step-by-step reasoning process, and wrap the final count in
\boxed{}.

How many people are in this image?
```

Answer: $\boxed{6}$

Incorrect Rollout

```
To determine the number of people in the image, I'll follow these
steps:

1.  Identify all individuals present in the picture.
2.  Count each person individually.
3.  Sum up the total count.

Step 1:  Identifying individuals
- There are five people standing together in the image.

Step 2:  Counting individuals
- The first person on the left is wearing a gray t-shirt with
"FITCHBURG STATE" written on it.
- The second person is wearing a light blue button-up shirt and khaki
pants.
- The third person is holding an award and wearing a black dress.
- The fourth person is wearing a yellow plaid shirt with gray pants.
```

```
- The fifth person is wearing a pink jacket over a black blouse and
white pants.
Step 3:  Summing up the total count
There are five distinct individuals visible in the image.
Therefore, the final count is:  \boxed{5}
```

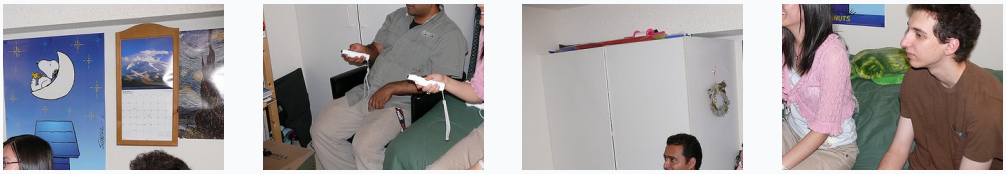
Revised Response

```
To determine the number of people in the image, I'll follow these
steps:
1.  Identify all individuals present in the picture.
2.  Count each person individually.
3.  Sum up the total count.
Step 1:  Identifying individuals
- There are six people standing together in the image.
Step 2:  Counting individuals
- The first person on the left is wearing a gray t-shirt with
"FITCHBURG STATE" written on it.
- The second person is wearing a light blue button-up shirt and khaki
pants.
- The third person is holding an award and wearing a black dress.
- The fourth person is wearing a yellow plaid shirt with gray pants.
- The fifth person is wearing a pink jacket over a black blouse and
white pants.
- The sixth person is wearing a light patterned outfit.
Step 3:  Summing up the total count
There are six distinct individuals visible in the image.
Therefore, the final count is:  \boxed{6}
```

G.3 JIGSAW

Figure 17 shows a Jigsaw example. The four tiles are the model inputs; the reconstructed image is shown only to aid reading and is not part of the prompt. The unaided rollout proposes the identity permutation `[[1, 2], [3, 4]]`. After rewriting the output for revision, the model recovers the verified ordering `[[3, 1], [2, 4]]`.

Model Inputs (Tile 1–4)



Tile 1 | Tile 2 | Tile 3 | Tile 4

Reconstructed Image (Illustration Only)



Figure 17: A demonstration of Jigsaw inputs and reconstructed image. Top: the four shuffled tiles fed to the model. Bottom: the original photograph reconstructed from the ground-truth ordering.

Example From Jigsaw

Problem:

```
You are given 4 image tiles, labeled Tile 1 to Tile 4 in the order
they are presented.  These tiles are the four quadrants of one
original photograph, cut into a 2x2 grid and then randomly shuffled.

Your task is to reassemble the original image.  Analyze the tiles
carefully:
- Match colors, textures, lighting and gradients across tile borders.
- Trace objects, lines and edges that are cut apart and must continue
between adjacent tiles.
- Use scene priors:  sky / ceiling / distant background usually
belongs to the top row, ground / floor / foreground to the bottom
row.

Think step by step, then give the final answer as a 2x2 matrix
wrapped in \boxed{}.  Row 1 of the matrix is the top row of the
```

original image (left cell first), row 2 is the bottom row. Each cell holds the label number of the tile that belongs at that position.

For example \boxed{[[3, 1], [4, 2]]} means Tile 3 is the top-left quadrant, Tile 1 the top-right, Tile 4 the bottom-left and Tile 2 the bottom-right.

Now, write your response as one concise paragraph, no longer than 200 words.

Answer: [[3, 1], [2, 4]]

Incorrect Rollout

By analyzing the images, I can see that the top row likely contains the sky/ceiling elements, while the bottom row has ground/floor elements. The first image shows a wall with posters and a calendar, which could be part of the sky/ceiling row. The second image depicts two people holding game controllers, which suggests an indoor setting, possibly the ground/floor. The third image shows a man looking up, and the fourth image shows a person sitting on a bed. The fourth image seems to belong to the ground/floor row due to the bed and the person sitting on it. By matching the colors and textures, the second image also appears to be from the ground/floor row because it features the same type of indoor setting as the fourth image. Thus, we can conclude that the top row consists of the first and second images, and the bottom row consists of the third and fourth images. Therefore, the reassembled image would be \boxed{[[1, 2], [3, 4]]}.

Revised Response

Tile 3 is the top-left because the wardrobe/cabinet background and seated man continue vertically. Tile 1 is the top-right because the cropped heads at the bottom of Tile 1 continue into the people sitting on the bed in Tile 4. Tile 2 is the bottom-left because the man looking up in Tile 2 connects to the seated woman in Tile 4 through the shared edge. Tile 4 is the bottom-right because the person sitting on the bed in Tile 4 is the same as the person in Tile 2. Therefore, the reassembled image would be \boxed{[[3, 1], [2, 4]]}.

H REVISION PROMPT TEMPLATES

The demonstrations above use a revision stage that edits an incorrect rollout against an expert reference. Below we list the three task-specific prompt templates in full.

H.1 CLOCK READING

Clock Reading Revision Template

```
You are an expert at reading analogue clocks.  The dial has three
hands:  a thin RED second hand, a long BLACK minute hand, and a short
thick BLACK hour hand.
Below are the image, the question, the correct time, a reference
answer, and the answer you wrote earlier.  Revise your earlier answer
so that it is correct, changing as little of it as possible.
## Rules
- Make the SMALLEST edit that makes the answer correct.  Keep your
earlier answer's wording, structure, order of the hands and overall
length wherever it is already consistent with the correct time;
rewrite only the spans that conflict with it.  If a sentence is
already right, copy it verbatim.
- If your earlier answer is already correct, reproduce it unchanged.
- Do not add new sections, headings, summaries, apologies or
commentary that your earlier answer did not have, and do not delete
parts that were already correct.
- Show the reasoning first, hand by hand, then close with the time
wrapped in \boxed{}.  A reply that jumps straight to the boxed time is
rejected.
- The time must be exactly {gt_time}, in that digit form (03:45:20 |
never 3:45:20, 15:45:20 or 03:45).
- Write it as your own reading of the image:  never mention the
earlier answer, the correct time or the reference answer, and never
reuse their wording.
## Question
{question}
## Correct time
{gt_time}
## Reference answer (style only)
{reference_answer}
## Your earlier answer
{previous_output}
---
Output only the revised answer:  the complete reading of the dial,
keeping as much of your earlier answer as possible, ending with the
time in \boxed{}.
```

H.2 COUNTING

Counting Revision Template

```
You are an expert visual counting assistant.  Your earlier answer
reached the wrong total ({wrong_count}).  Repair it with the fewest
edits that make it genuinely correct.
You are given the image, the counting question, the verified correct
total, a reference answer written by someone else, and your earlier
attempt.
```

```
## How to revise
1. Look at the image again and locate every object the question asks
about.
2. Compare them against your draft's items: which objects are
missing, which are counted twice, which belong to the wrong category,
which were merged into a single line.
3. Patch exactly those items | add, delete or re-describe them | and
renumber if needed.
4. Recount your own list. It must account for exactly {gt_count}
objects before you finish.
## Core principle: change as little as possible
- Keep your draft's opening sentence, item order, numbering and
wording wherever they already hold.
- Edit only the spans that are wrong; never rewrite, reorder or polish
parts that are already fine.
- Output the whole draft with the patches applied, not just the lines
you changed. If the draft had no enumeration, write one.
## What "fewest edits" does not mean
- The enumeration is what went wrong, so the enumeration is what you
must repair. Changing only the final number and leaving the list
untouched is INVALID.
- Your list and your total must agree. If the list accounts for four
objects, writing {gt_count} on the last line is a contradiction, not a
fix.
## Do not copy the reference answer
- The reference answer exists only to show you WHICH objects are in
the image. It is a diagnostic aid, never a template.
- Never reuse its sentences, phrasing, item order or opening line.
Describe each object in your draft's own voice and vocabulary.
- Rewriting your answer from the reference wastes the sample: a
response that substantially copies it is discarded downstream even
when the total is right.
## Hard requirements
- Include the full enumeration, one line per object. An answer that
is only the final line is rejected.
- Every item must describe something actually visible | position,
color, appearance or another distinguishing detail. Never invent an
object to reach the total.
- If objects are too crowded to separate one by one, describe them as
a group and state that group's subtotal.
- Write as if you had read the image yourself: never mention the
reference answer, the verified total, your earlier attempt, or this
instruction block.
- Close with the total on its own last line, in exactly this form:
Final count: \(\boxed{N}\)
## Counting question
{question}
## Verified correct total
{gt_count}
## Reference answer (to identify the objects only; never quote it)
{reference_answer}
## Your earlier attempt (the draft to edit; it answered {wrong_count})
{wrong_output}
---
Output only the repaired answer: your edited draft, with the
enumeration lines followed by the final count line. No preamble, no
notes on what you changed.
```

H.3 JIGSAW

Jigsaw Revision Template

```
You are an expert at reassembling shuffled image tiles.  You are
given the four tiles of one photograph, the question that was asked
about them, the verified correct arrangement, a reference answer, and
a previous INCORRECT attempt made by a model.

Your task is to REPAIR that attempt:  fix what is wrong in its
reasoning and in its final answer, while keeping whatever it already
got right.  Whatever the attempt looks like, your own answer must
always be a complete step-by-step derivation | never a bare matrix.

## Required output shape

Output a full, self-contained solution that reasons step by step and
ends with the final matrix:

1.  One step per position:  say which tile goes in the top-left, the
top-right, the bottom-left and the bottom-right, and for each one
give the specific visual evidence that puts it there (which object,
edge, line, horizon or texture is cut apart and continues into which
neighbouring tile, or which scene cue such as sky/distant background
vs ground/foreground fixes its row).
2.  One step that confirms the left/right order inside each row,
since a row can be right while its two tiles are swapped.
3.  The final line, in exactly this form:

Therefore the correct reconstruction is \(\boxed{[[a, b], [c, d]]}\)

## Repair the reasoning | never append to it

The previous attempt reached the wrong arrangement because parts of
its reasoning are wrong.  Fix that reasoning; do not work around it.

- Correct every claim that led to the wrong answer.  A sentence that
puts a tile in the wrong quadrant, that says a seam, object or
texture continues where it does not, or that concludes a row/column
order disagreeing with the final matrix, must be rewritten | not
kept.
- NEVER keep the attempt's wrong conclusion and bolt the right answer
onto the end.  Appending a line like "Therefore the correct
reconstruction is ..." after the attempt's original wrong conclusion
is the worst possible outcome:  it leaves the answer arguing for one
arrangement while stating another.
- Your output must contain EXACTLY ONE \boxed{}, the final one.
Remove the attempt's old matrix; never leave two matrices in the
answer.
- Do not just copy the attempt's text and change the digits in the
last matrix.  If the reasoning above the matrix still supports the
old arrangement, the answer is wrong even though the matrix is right,
and it will be rejected.
- Before you finish, read your answer back once and check it against
{gt_readable}:  every quadrant you name, every piece of evidence and
the final matrix must agree with each other and with that arrangement.
Fix anything that does not.

## Hard requirements

- THE STEP-BY-STEP REASONING IS MANDATORY, no matter what the previous
attempt looks like.  An answer that is only the final matrix, or that
has a couple of sentences with no visual evidence, is INVALID and
will be rejected.  Always write all four position steps plus the
left/right check.
- The final matrix must be exactly {gt_matrix}, i.e.  {gt_readable}.
- When the previous attempt has no reasoning to preserve | for
instance it is a bare matrix | there is simply nothing to keep:
derive the whole answer yourself from the tiles.  Answering with
```

```
another bare matrix is the worst possible output.  When it does have
reasoning, your answer must be at least as detailed as it is.
- Keep what already works:  reproduce word for word those sentences of
the previous attempt that are consistent with the correct arrangement,
and preserve its wording, order and style.  Change what is wrong, and
nothing else.
- Name the tiles explicitly as "Tile 1" ...  "Tile 4" and cite only
evidence actually visible in the tiles | never invent an object, an
edge or a marking, and never pad the text to make it longer.
- State each tile's position the way the final matrix does.  Every
"Tile N is the top-left / top-right / bottom-left / bottom-right"
claim in your text must match {gt_readable} exactly; a single
mismatched quadrant makes the whole answer invalid.
- DO NOT COPY THE REFERENCE ANSWER, and do not reuse any wording from
these instructions.  The reference answer is shown only so you can
check which visual evidence is true; where it and the previous
attempt say the same thing differently, keep the previous attempt's
wording.
- Write as if the attempt had been right the first time.  NEVER
mention the reference answer, the verified arrangement, the previous
attempt or the fact that anything was corrected; avoid words such as
"reference", "ground truth", "hint", "the given arrangement",
"previously" and "instead of".
## Jigsaw Question
{question}
## Verified Correct Arrangement
{gt_matrix} ({gt_readable})
## Reference Answer (known correct; consult it ONLY to check which
evidence is true | never copy from it)
{reference_answer}
## Model's Incorrect Attempt (it answered {wrong_matrix}) | repair
this text:  correct its faulty reasoning, keep its correct parts, and
do not merely append an answer to it
{wrong_output}
---
Output ONLY the repaired answer itself:  the step-by-step derivation
covering all four positions and the left/right check, ending with a
single `\boxed{}` line whose reasoning fully supports it.  No
preamble, no diff, no list of what you changed.
```

I Concise Revision Prompt Templates

Below we present more concise and abstract revision templates, without the fine-grained rule constraints of Appendix H. Empirically, this style of prompt underperforms on smaller models (3B and 7B) but works well on larger ones (32B and 72B), indicating that revision competence improves with model scale.

I.1 Clock Reading

Clock Reading revision template (concise)

```
You are an expert at reading analogue clocks.  The dial has three
hands:  a long, thin RED second hand, a long BLACK minute hand, and a
short, thick BLACK hour hand.
Below are the image, the question, a reference reasoning trajectory,
and a model's WRONG attempt.  Compare the wrong attempt with the
```

```
correct reference trajectory, identify the specific mistakes in the
wrong attempt, and revise it to fix those mistakes.  Your task is
only to repair the wrong attempt.  Use the reference trajectory
solely as a source of information for correcting errors; do not copy
it as a replacement for the wrong attempt.
## Rules
- The time must be written in the zero-padded 12-hour HH:MM:SS format
(for example, 03:45:20 | never 3:45:20, 15:45:20, or 03:45).
- Correct the specific mistakes made in the wrong attempt.
## Question
{question}
## Reference trajectory
{reference_answer}
## Wrong attempt
{wrong_output}
---
Now write the repaired version of the wrong attempt.  Minimize the
edit distance from the original attempt:  preserve its wording and
structure as much as possible, changing only what is necessary to
correct the errors.
```

I.2 JIGSAW

Jigsaw revision template (concise)

```
You are an expert at reassembling shuffled image tiles.  The four
tiles above are the quadrants of one photograph, cut into a 2x2 grid
and then shuffled.
Below are the tiles, the question, a reference answer, and a model's
WRONG attempt.  Compare the wrong attempt with the correct reference
answer, identify the specific mistakes in the wrong attempt, and
revise it to fix those mistakes.  Your task is only to repair the
wrong attempt.  Use the reference answer solely as a source of
information for correcting errors; do not copy it as a replacement
for the wrong attempt.
## Rules
- The final answer must be a 2x2 matrix wrapped in \boxed{}, written
as \(\boxed{[[a, b], [c, d]]}\), and your answer must contain no other
matrix.
- The wrong attempt put the tiles in the wrong order, so the sentences
that placed them must change too.  Rewrite every sentence that puts a
tile in a quadrant it does not occupy, or that claims something
continues between tiles that are not neighbours.  Never leave a
sentence arguing for one arrangement while your matrix states
another.
- Apart from those sentences, change as little as possible:  keep the
attempt's own wording, sentence order, level of detail and overall
length.
- Present it as your own reading of the tiles:  never mention the
attempt or the reference answer, and do not say that anything was
corrected.
## Question
{question}
## Reference answer
{reference_answer}
## Wrong attempt
```

```
{wrong_output}
---
Now write the repaired version of the wrong attempt.  Minimize the
edit distance from the original attempt:  preserve its wording and
structure as much as possible, changing only what is necessary to
correct the errors.
```